User as Code: Executable Memory for Personalized Agents

Bojie Li
Pine AI

Abstract

Personalized AI agents today rely on retrieval over unstructured text, knowledge graphs, or flat fact stores. These "bag-of-facts" representations recall well but struggle to resolve conflicting reports, aggregate over many records, and enforce logical constraints. We introduce **User as Code** (UaC), a paradigm in which an agent's model of a user is a modular, version-controlled software project of typed Python dataclasses and executable constraints. The core contribution is a *two-phase pipeline*—append-only fact extraction followed by periodic LLM-driven structuring into typed code—so an interpreter can run constraints deterministically and surface pre-computed alerts. The shape (append log + periodic checkpoint) borrows from database systems; the novelty is using typed code as the checkpoint medium for LLM memory.

We evaluate UaC against five memory systems on a shared Gemini 3 Flash backbone. UaC reaches **78.8%** on the full 10-conversation LOCOMO benchmark (600 QAs), within 1.0pp of the 79.8% full-context upper bound (McNemar p=0.65), and **83.0%** on the full 500-question LongMemEval—statistically tying MemMachine (84.8%, p=0.33) and significantly ahead of EverMemOS, Hindsight, A-MEM, and Mem0 ($p < 0.005$). A new 100-case analytical-inference benchmark separates representations by class: retrieval-only memory collapses on aggregation (Mem0 6%, MemMachine 43%) while code-executable representations stay near 100% (UaC 99%, Full Context+REPL 100%). Token instrumentation shows UaC's one-time structuring pays back after $\sim$3 repeated queries and costs $\sim$15$\times$ less than reloading raw records over 100 queries. On a 60-scenario Active Service benchmark, UaC's constraint pipeline reaches 100% alert detection on standard scenarios and 85% on hard multi-step-arithmetic scenarios. SOTA comparisons (MemMachine, EverMemOS, Hindsight) use minimal-faithful same-backbone reimplementations (Appendix A); their published numbers serve as architecture ceilings, not direct competitors.

Code: https://github.com/19PINE-AI/user-as-code
Website: https://01.me/research/user-as-code

1 Introduction

Personalized AI agents need a *user memory*: a model of the user that accumulates across sessions, not a transcript of every utterance. User memory tasks fall into three tiers: (1) **Basic Recall**—retrieving unambiguous facts; (2) **Multi-session Retrieval**—reasoning over scattered, possibly conflicting information; and (3) **Active Service**—synthesizing information to provide anticipatory, unsolicited help. Existing benchmarks [Maharana et al., 2024, Wu et al., 2025, Hu

et al., 2025a] test the first two but miss the *initiation asymmetry* central to (3): the system must generate alerts *without being asked*, triggered by state changes rather than queries.

Current memory systems—vector databases, JSON stores, knowledge graphs, flat fact extractors—treat memory as a loose collection of facts. This creates two problems. Conflicting facts ("I love cilantro" vs. "I actually hate cilantro") coexist unresolved without version history. And even the most advanced formats cannot express executable rules like "if passport expires within 180 days of travel date, raise an alert."

Thesis. Free-text and fact-store formats separate *representation* from *verification*. Any representation a Python interpreter can read directly—typed objects, JSON dicts, even strings with a parser—collapses the two: `passport.expiry_date = date(2025, 2, 18)` can be stored *and* compared in one medium (Listings 6–7). The analytical benchmark confirms this: a Full Context + Python REPL baseline over raw JSON matches UaC on aggregate queries (Section 4.3), so "code-as-representation" is not by itself a novel claim. *UaC's contribution is the two-phase pipeline that converts unstructured conversation into the code-readable representation in the first place*—without it, every aggregate query pays LLM parsing cost over noisy dialogue. The pipeline shape (append-only log + periodic checkpoint) borrows from database systems; the novelty is applying it to LLM memory with typed Python as the checkpoint medium.

```
# Typed user state (excerpt, auto-generated by Phase 2):
passport = PassportInfo(number="AB1234567",
                        expiry_date=date(2025, 2, 18))
trips = [Trip(destination="Tokyo",       departure_date=date(2025, 1,
    15),
              is_international=True),
         Trip(destination="Mexico City", departure_date=date(2025, 3,
             10),
              is_international=True),
         Trip(destination="Portland",    departure_date=date(2025, 4,
             22),
              is_international=False)]

# (a) Recall -- one attribute access:
>>> passport.number
'AB1234567'

# (b) Analytical aggregate -- one-line expression over the collection:
>>> sum(1 for t in trips if t.is_international
                         and t.departure_date.year == 2025)
2

# (c) Constraint -- boolean check; interpreter fires the alert:
>>> (passport.expiry_date - trips[0].departure_date).days >= 180
False                # 34 days; passport renewal needed before Tokyo
```

Listing 1. The three capability tiers UaC provides, all against the same auto-generated typed state. (a) Recall is attribute access—what every memory system gives you. (b) Analytical aggregate is a one-line expression—where retrieval-based systems collapse (Section 4.3). (c) Constraint is a boolean check the interpreter fires on every state change—the basis of the proactive alerts in Section 4.4.

Three things memory-as-code unlocks. Listing 1 shows the three capability tiers, all against the same typed state, all one-line Python. (a) **Recall** is attribute access. (b) **Analytical inference** is a list comprehension—trivial in Python, structurally lossy under top-k retrieval (UaC 99% vs. MemMachine 43%, Section 4.3). (c) **Active service** is a boolean check over typed dates—one comparison the interpreter runs deterministically at every state change. Listings 2–5 expand (c) into a life-safety walkthrough where the relevant facts arrive ten months apart.

A deeper example: cross-session active service. The check in Listing 1(c) is small because the relevant facts sit in one state object. The harder case is when they arrive in different conversations and only meet inside the schema. Listings 2–5 show this end-to-end. Two utterances ten months apart (Listing 2) distill into an append-only fact list and a typed Python state (Listing 3); they never co-occur in any retrieval window, but `drug_class="penicillin"` on both an `Allergy` and a `Medication` is a deterministic boolean check. The coding agent writes a 12-line constraint (Listing 4); the interpreter runs it; the alert (Listing 5) appears in the next session's manifest before the user asks anything. Free-text retrieval would need to either know that amoxicillin is a penicillin (uncertain under retrieval pressure) or rank a year-old allergy turn highly against a sinus-infection query—neither is reliable.

```
SESSION 12 -- DATE: 2024-03-01
Jessica: Saw Dr. Park about my seasonal allergies, she put me on
         cetirizine 10mg. Reminder -- I'm DEATHLY allergic to
         penicillin, anaphylaxis. Last time I almost died.

SESSION 47 -- DATE: 2025-01-10  (ten months later)
Jessica: Quick log: Dr. Robert Chen prescribed amoxicillin 500mg,
         three times a day for ten days, for the sinus infection.
```

Listing 2. Two conversation turns, ten months apart.

```python
# Phase 1 -- append-only fact list (never overwritten)
facts = [
    "[2024-03-01] User has SEVERE penicillin allergy (anaphylaxis)",
    "[2024-03-01] Dr. Park prescribed cetirizine 10mg daily",
    "[2025-01-10] Dr. Chen prescribed amoxicillin 500mg, 3x/day, "
    "10 days, sinus infection",
]

# Phase 2 -- structured Python (regenerated from full fact corpus)
medical_profile = MedicalProfile(
    allergies=[Allergy(allergen="Penicillin", severity="severe",
        reaction="Anaphylaxis", drug_class="penicillin")],
    current_medications=[
        Medication(name="Cetirizine", drug_class="antihistamine",
            dosage="10mg", prescriber="Dr. Park",
            start_date=date(2024, 3, 1)),
        Medication(name="Amoxicillin", drug_class="penicillin",
            dosage="500mg", prescriber="Dr. Chen",
            start_date=date(2025, 1, 10)),
    ],
)
```

Listing 3. Phase 1 facts and Phase 2 typed state. The shared `drug_class="penicillin"` key anchors the cross-session link.

```python
def check_drug_allergy(profile: MedicalProfile) -> list[Alert]:
    alerts = []
    for med in profile.current_medications:
        for allergy in profile.allergies:
            if med.drug_class == allergy.drug_class:
                alerts.append(Alert(
                    severity="critical", domain="health",
                    message=(f"DRUG-ALLERGY CONFLICT: {med.name} "
                             f"({med.drug_class}-class), prescribed "
                             f"{med.start_date} by {med.prescriber}; "
                             f"patient has {allergy.severity} "
                             f"{allergy.allergen} allergy "
                             f"({allergy.reaction})."
    return alerts
```

Listing 4. The constraint the coding agent writes once, against the typed state. The interpreter executes it deterministically – no LLM at check time.

```
[CRITICAL/health]  DRUG-ALLERGY CONFLICT: Amoxicillin
(penicillin-class), prescribed 2025-01-10 by Dr. Chen;
patient has severe Penicillin allergy (Anaphylaxis).
```

Listing 5. Alert surfaced in `ACTIVE_ALERTS` at the start of every subsequent session. No retrieval, no question asked – the constraint fired the moment the state was updated.

Concretely, we model user memory as a self-evolving software project—typed dataclasses, domain packages, executable constraints—driven by a two-phase architecture. Append-only fact extraction never loses information; periodic code structuring organizes facts into typed Python. Together they enable a generate–verify–review loop: the coding agent writes constraints, the interpreter verifies them deterministically, and results surface as pre-computed alerts in a compact manifest. Our contributions:

- **A two-phase memory architecture** reaching 78.8% on the full 10-conversation LOCOMO benchmark (600 QAs), within 1.0pp of the full-context upper bound (79.8%, McNemar p=0.65), and significantly ahead of same-backbone SOTA reimplementations: MemMachine 72.7%, Hindsight 69.7%, EverMemOS 55.5% (all $p < 0.005$). The same architecture reaches 100% proactive alert detection on 40 standard Active Service scenarios (85% on 20 hard scenarios).
- **An Active Service benchmark** testing proactive alerting across 60 scenarios in 5 constraint categories, the first to measure memory-triggered alerts initiated without a user query.
- **A new analytical-inference benchmark** (100 cases, 10 record types, $N \in \{20, 50, 100, 200, 500\}$) exposing a structural gap between code-executable representations (UaC 99%, FC+REPL 100%) and retrieval-based ones (MemMachine 43%, Mem0 6%). Token instrumentation shows UaC's structuring pays back after $\sim$3 queries against the same user state and amortizes to $\sim 15\times$ cheaper than reloading raw records.

- **Three orthogonal ablations** attributing the architecture's gains: (i) the two-phase separation (append-only + periodic restructuring) is the decisive recall mechanism (+19pp over incremental code); (ii) a leave-one-out channel study shows the typed state is roughly neutral for LOCOMO recall (-1.3pp, p=0.67) but decisive for the analytical and constraint tiers retrieval cannot serve; (iii) Modularity/Progressive Disclosure reduces prompt-token cost 14.9$\times$ with no accuracy loss on 500-record states.
- **Cross-LLM and cross-judge robustness checks.** Replacing Gemini 3 Flash with GPT-5.4 throughout the pipeline yields a statistical tie on a 120-QA LOCOMO subset (p=0.82). Rejudging all 7,700 predictions under Claude Opus 4.7 preserves the ranking with $\kappa \geq 0.74$ on every system.
- **Open-source implementation** with same-backbone comparisons against five memory systems (Mem0, A-MEM, MemMachine, EverMemOS, Hindsight) plus full-context baselines on LOCOMO, LongMemEval, Active Service, Analytical Inference, and Modularity.

2 Related Work

LLM agent memory has grown rapidly in 2025–2026, with 20+ systems, 15+ benchmarks, and several surveys [Hu et al., 2025b, Du, 2026, Yang et al., 2026]. We organize related work around three gaps UaC addresses.

2.1 Gap 1: Representation Separates Storage from Verification

Existing memory systems store user state in formats that cannot be directly executed. **Flat fact systems** like Mem0 [Chhikara et al., 2025] extract atomic facts with a CRUD lifecycle. **Structured-note systems** like A-MEM [Xu et al., 2025] use Zettelkasten-inspired notes with dynamic linking; ENGRAM [Patel and Patel, 2025] shows that careful typing plus dense retrieval can match more complex architectures. **Knowledge-graph approaches** like Zep/Graphiti [Rasmussen et al., 2025] build temporal KGs with bitemporal modeling; MAGMA [Jiang et al., 2026] adds multi-graph traversal. **Hybrid architectures** like Hindsight [Latimer et al., 2025] maintain four logical networks (91.4% on LongMemEval); MemMachine [Wang et al., 2026] stores entire episodes (91.7% on LOCOMO). **OS-inspired systems** like MemGPT/Letta [Packer et al., 2023] and MemOS [Li et al., 2025] treat memory paging as an OS primitive: the agent edits text-based "main context" and "recall storage" pages through a custom API (Letta's `core_memory_replace / archival_memory_insert`), but the page contents remain unstructured text—the OS is around the memory, not in it. **Agent-editable instructions** are taken furthest by LangMem [LangChain Team, 2025], which lets the agent rewrite its own system-prompt directives over time; this is the closest existing system to "executable memory," but the medium remains natural language interpreted by the LLM at read time, not typed objects an interpreter can run.

How UaC differs. UaC commits to typed Python for the *state itself*, not just for instructions or paging machinery. Concretely: (i) user state is a directory of Python dataclass instances, not text pages—an interpreter indexes, groups, and computes over it without reparsing; (ii) persistent "rules" are Python functions over that state (e.g., `check_drug_allergy(profile) -> list[Alert]`), not natural-language directives the LLM re-interprets each call; (iii) MemGPT-style paging and LangMem-style instruction editing remain available on top—each domain is

a module, each constraint a callable—but neither is required for the headline capabilities. The closest "state-as-code" work outside the memory community is constraint-as-code in policy languages (e.g., Rego/OPA); UaC differs in that schemas, instances, and constraints are all LLM-generated and evolve with the user, not human-authored and frozen.

2.2 Gap 2: No Benchmark Tests Proactive Memory-Triggered Alerting

LOCOMO [Maharana et al., 2024] provides 1,986 QA pairs; LongMemEval [Wu et al., 2025] tests 500 questions across six question types. LoCoMo-Plus [Li et al., 2026] tests implicit constraints; MemoryArena [He et al., 2026] shows LOCOMO-saturated agents fail in agentic settings. ProAgentBench [Tang et al., 2026] tests proactive task assistance but not memory-triggered alerting. PersistBench [Pulipaka et al., 2026] identifies memory safety risks. To our knowledge no existing benchmark tests the *initiation asymmetry* central to Active Service—the system must produce alerts in the absence of a user query, triggered by state changes alone—which motivates the new Active Service benchmark in Section 4.4.

2.3 Gap 3: Memory Operations Are Not Yet Optimized End-to-End

Memory-R1 [Yan et al., 2025] trains memory policies with GRPO (+48% F1). AgeMem [Yu et al., 2026] learns proactive summarization through progressive RL. Mem-alpha [Wang et al., 2025] demonstrates generalization to 400K+ tokens. Cognitively inspired systems include FadeMem [Wei et al., 2026] (biologically inspired forgetting), LightMem [Fang et al., 2025] (Atkinson-Shiffrin model), and EverMemOS [Hu et al., 2026] (engram-inspired lifecycles, 93.05% on LOCOMO). All train over custom memory APIs; UaC's file-system-based architecture makes memory operations native file actions, which is directly compatible with the same RL machinery.

3 Methodology: The User as Code Architecture

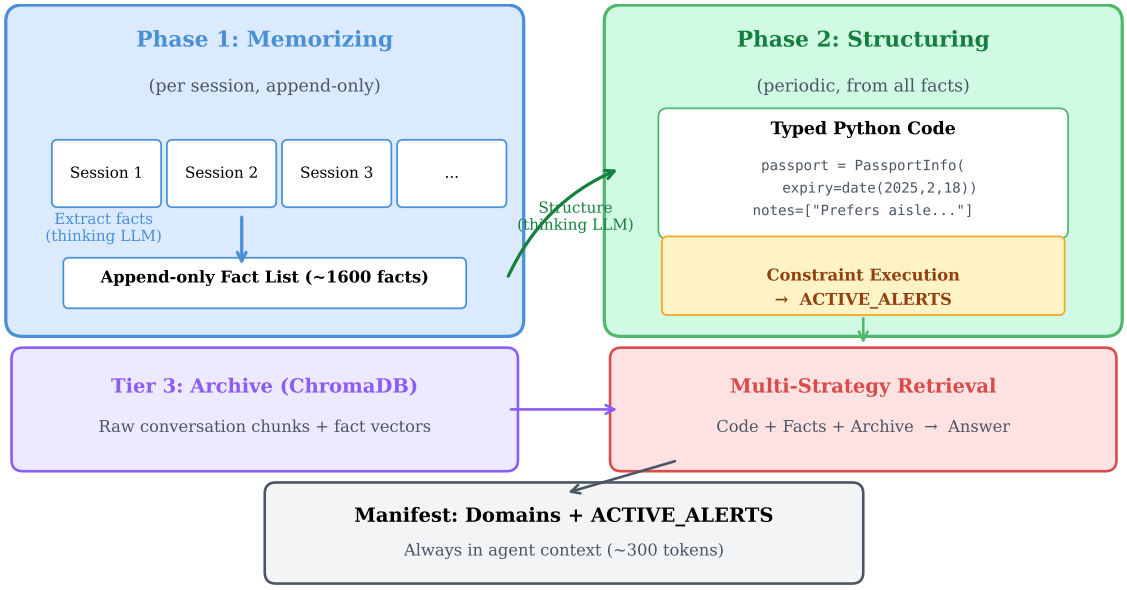

Figure 1. User as Code two-phase architecture. Phase 1 extracts facts from each session (append-only, never overwritten). Phase 2 periodically structures all accumulated facts into typed Python dataclasses. Multi-strategy retrieval combines structured code, fact vectors, and raw archive. Constraint execution produces pre-computed alerts surfaced in the manifest.

3.1 What "User as Code" Looks Like

Listings 6–8 show the three artifacts of a UaC project, taken verbatim from our reference implementation. The state is a typed dataclass instance, so date arithmetic like `(passport.expiry_date - trip.departure_date).days` is one line of Python (Listing 6). A constraint over that state is itself Python (Listing 7)—same medium, same interpreter, no DSL. Pre-computed alerts surface in a compact manifest (Listing 8) loaded at the start of every conversation, so the agent sees outstanding issues with no retrieval. Listings 2–5 show the same pattern for health; Appendix F traces a verbatim LOCOMO conversation end-to-end.

```python
from datetime import date
from .schema import PassportInfo, Trip

passport = PassportInfo(
    number="AB1234567", country="US",
    issue_date=date(2015, 2, 18),
    expiry_date=date(2025, 2, 18),
    full_name="Jessica Marie Thompson",
)

trips = [
    Trip(destination="Tokyo", country="JP",
         departure_date=date(2025, 1, 15),
         return_date=date(2025, 1, 25),
         flight_number="JAL-9823",
```

```
          is_international=True),
    Trip(destination="Mexico City", country="MX",
          departure_date=date(2025, 3, 10),
          return_date=date(2025, 3, 17),
          flight_number="AA-4561",
          is_international=True),
]
```

Listing 6. Excerpt from `domains/travel/state.py` of a UaC user project. Generated automatically from the append-only fact list.

```
from datetime import timedelta
from domains.travel.state import passport, trips

def check():
    alerts = []
    for trip in trips:
        if not trip.is_international:
            continue
        days = (passport.expiry_date - trip.departure_date).days
        if days < 180:
            alerts.append({
                "severity": "critical",
                "domain":   "travel",
                "message": (
                    f"Passport {passport.number} expires "
                    f"{passport.expiry_date} -- only {days} days "
                    f"before {trip.destination} on "
                    f"{trip.departure_date}."),
            })
    return alerts
```

Listing 7. Excerpt from `constraints/travel_readiness.py`. The constraint reads the typed state and returns structured alerts. The interpreter runs it deterministically; no LLM is required at check time.

```
DOMAINS = {
    "travel":   {"path": "domains/travel",
                 "summary": "Passport AB1234567 (exp 2025-02-18), "
                            "2 upcoming trips"},
    "health":   {"path": "domains/health",
                 "summary": "Allergies: peanuts, penicillin. "
                            "Rx: cetirizine, amoxicillin"},
    "finance":  {"path": "domains/finance",
                 "summary": "Chase Sapphire (DTI 0.31), Roth IRA "
                            "($42K), Q1 tax estimate due 2025-04-15"},
    "work":     {"path": "domains/work",
                 "summary": "Senior PM at Acme; 4 direct reports; "
                            "Q1 OKR review 2025-03-28"},
    "people":   {"path": "domains/people",
                 "summary": "Spouse Mark, sister Anna, 12 close "
                            "contacts; Mark birthday 2025-06-12"},
    "household":{"path": "domains/household",
```

```
                    "summary": "Mortgage refi rate-lock expires "
                               "2025-02-28; HVAC warranty until 2027"},
}

ACTIVE_ALERTS = [
    {"severity": "critical", "domain": "travel",
     "message": "Passport AB1234567 expires 2025-02-18 -- "
                "only 34 days before Tokyo on 2025-01-15."},
]
```

Listing 8. The manifest, always loaded into the agent's context. `ACTIVE_ALERTS` is overwritten by the constraint runner; the agent surfaces them proactively without retrieval.

3.2 Two-Phase Architecture

Our key architectural insight, validated by ablation (Section 4.5), is that memorizing and structuring must be separate concerns:

Phase 1 — Memorizing (per session, append-only). An LLM extracts every individual fact as a flat string from each session (Gemini 3 Flash, thinking budget 8192). Facts are appended to a running list—never overwritten, never deleted—and relative dates are resolved to absolute dates against the session timestamp. This produces ∼50 facts/session (∼950 for a 19-session LOCOMO conversation). Facts are indexed in ChromaDB.

Phase 2 — Structuring (periodic, from all facts). An LLM regenerates the entire structured Python from the accumulated fact list (thinking budget 16,384), organizing facts into typed dataclasses: `date()` for dates, typed lists for collections, `notes: list[str]` for hard-to-type facts. Because the code is regenerated fresh from the complete corpus—not edited incrementally—no information is lost in the fact-to-structure transition. Code size scales with distinct facts (∼7K characters for a 19-session LOCOMO conversation; 50–55K for the analytical benchmark at N=500).

Archive (raw conversation). Raw conversation chunks are also indexed in ChromaDB as a retrieval fallback for direct-quote queries.

Phase 2 cost scaling. Wall time stays at 30–40 seconds across $\{19, 50, 100, 200\}$-session corpora, and cost flattens at ∼$0.07 once our 500K-character input cap engages around n=100 sessions (Appendix E)—a known limitation that motivates hierarchical structuring at production scale.

3.3 Design Principles

Two principles are evaluated directly in this paper:

1. **Separation of Structure and Data.** Schema (dataclasses) separated from state (instances). Validated by the ablation in Section 4.5.

2. **Unified Representation and Verification.** Code serves as both storage and verification medium. Validated by Active Service (Section 4.4) and Analytical Inference (Section 4.3).

Three further principles guide production deployment but are not isolated by the experiments:

3. **Modularity by Life Domain.** Memory partitioned into independent domain packages (e.g., `travel/`, `health/`, `finance/`). Limits cross-domain leakage and supports selective loading at scale.

4. **Progressive Disclosure.** Compact manifest (∼200–300 tokens, always loaded) with on-demand domain loading. Keeps the prompt budget bounded as the user state grows.

5. **Agent-Native File System.** User project is a directory in the agent's workspace—no custom memory API needed.

3.4 The Generate-Verify-Review Loop

The core mechanism for Active Service:

1. **Generate:** The coding agent writes Python constraints against the code-represented state.

2. **Verify:** Execute in a sandbox—results are deterministic.

3. **Review:** The agent reviews results, decides to refine, persist, or incorporate.

When an ad-hoc check proves useful, the agent promotes it to a persistent constraint. Alerts surface in the manifest's `ACTIVE_ALERTS`, visible at the start of every future conversation. Constraints emerge autonomously from the LLM's reasoning.

3.5 Multi-Strategy Retrieval

At query time, retrieval composes three channels rather than picking one. Each channel addresses a different failure mode of the others: the structured-code channel can return a typed object but only for facts the structuring step has captured; the fact-vector channel covers the long tail of facts that did not make it into the typed schema; and the raw-archive channel preserves the exact conversational phrasing for queries that hinge on wording.

(1) Structured code state. The full `state.py` of the manifest-routed domain (or all domains, in the non-modular setting) is included verbatim in the prompt, truncated at 6,000 characters. The agent reads typed dataclass instances and the pre-computed `ACTIVE_ALERTS` list with no parse step.

(2) Fact-vector retrieval. A ChromaDB collection over the append-only fact list (one fact per record, ∼50 facts/session) is queried by cosine similarity against the question; the top 20 facts are appended. This recovers facts the structuring step compressed away or normalized to a different surface form.

(3) Archive retrieval. A second ChromaDB collection holds the raw conversation chunked at session granularity; the top 10 chunks by cosine similarity are appended. Used as a last-resort fallback for direct-quote questions.

The three channels are concatenated under fixed headers (`[STATE]`, `[FACTS]`, `[ARCHIVE]`) and passed to the answer LLM with no reranking; the prompt instructs it to prefer the structured channel on conflicts. The leave-one-out channel ablation in Section 4.6 shows the fact-vector and archive channels are individually significant on LOCOMO recall while the structured channel is roughly neutral for recall—it instead carries the analytical and constraint workloads.

4 Experiments and Evaluation

4.1 Setup

LLM and judge. Gemini 3 Flash (`gemini-3-flash-preview`) is used throughout with thinking enabled (answer-time budget 2048; no `max_output_tokens` cap). Evaluation is LLM-as-Judge with binary CORRECT/WRONG scoring (budget 256). The judge sees the question, gold answer, and prediction and is instructed to be generous: same-information-different-wording scores CORRECT; WRONG only when factually wrong or claiming unavailability when the gold exists.

Baselines. Five external memory systems plus a Full-Context upper bound (all raw transcripts in-prompt). *Production libraries:* Mem0 [Chhikara et al., 2025] (mem0ai 1.0.5; flat fact extraction) and A-MEM [Xu et al., 2025] (Zettelkasten notes). *Same-backbone reimplementations of recent SOTA systems* (Gemini 3 Flash + ChromaDB; fidelity statement in Appendix A): MemMachine [Wang et al., 2026] (sentence-level episodes, ± 3 contextual expansion), Hindsight [Latimer et al., 2025] (four-network fact graph, RRF fusion, cross-encoder rerank), EverMemOS [Hu et al., 2026] (MemCell$\rightarrow$MemScene consolidation, Foresight-boosted retrieval).

Benchmarks. All systems share the same evaluation data. LOCOMO [Maharana et al., 2024]: the full 10 conversations, 60 QAs each (600 total). LongMemEval [Wu et al., 2025]: the full 500-question benchmark (133 temporal-reasoning, 133 multi-session, 78 knowledge-update, 70 single-session-user, 56 single-session-assistant, 30 single-session-preference). Active Service: 40 standard + 20 hard scenarios.

Cross-LLM portability. To verify that gains do not depend on a particular code-generator's idiosyncrasies, we additionally re-run UaC with GPT-5.4 (OpenAI) substituted throughout (extraction, structuring, answer generation), keeping the Gemini judge for fair scoring against the Gemini run. The cross-LLM run uses a 2-conversation LOCOMO subset (120 QAs).

Which model runs where. To keep the comparison controlled, *answer generation and the LLM-as-Judge are Gemini 3 Flash for every system*, and retrieval in Mem0 and A-MEM is embedding-based (their default encoders over ChromaDB) and invokes no LLM at all. The only non-Gemini calls live *inside* the two production libraries' own write-time memory construction—Mem0's fact extraction/consolidation and A-MEM's note synthesis and linking—which default to OpenAI `gpt-4o-mini`; we keep that library default and route it through OpenRouter on a key with the `model.request` scope so it actually runs (an earlier draft had it failing silently). In other words, `gpt-4o-mini` never reads a question, ranks a retrieval, or writes an answer: it only distills each ingested session into the stored memories of those two libraries, and every accuracy this paper reports comes from an answer-time comparison held fixed at Gemini 3 Flash. The residual handicap from this weaker write-time model is one of the factors we isolate in the Mem0 diagnostic (Section 4.2). The cross-family Claude judge in Appendix C also uses OpenRouter; all Gemini calls use the Google API directly.

Statistical interpretation. Headline accuracies are proportions correct; at n=600/500 the 95% Wilson half-width is $\sim$3–4pp at the relevant accuracy range (per-row CIs in Tables 1–2). Because all systems answer the *same* questions, marginal CIs overstate pairwise uncertainty; we report McNemar's exact test on per-question correctness vectors for all comparisons. Headline result: UaC is statistically tied with Full Context at the top of both benchmarks, ahead of every competing memory system at α=0.05 on LOCOMO, and in a three-way top cluster with MemMachine and Full Context on LongMemEval (per-pair p-values in Tables 1–2).

4.2 Standard Benchmarks

Table 1. LOCOMO results on the *full* 10-conversation benchmark (60 QAs/conv, 600 total). LLM-as-Judge accuracy with thinking-enabled Gemini 3 Flash judge; 95% Wilson CIs reported per row. McNemar's exact test compares UaC vs. each row on the per-question correctness vector. UaC, Full Context, Mem0, and A-MEM use Gemini 3 Flash for answer generation; UaC (GPT-5.4) substitutes GPT-5.4 for cross-LLM portability. Recent SOTA systems are run as minimal-faithful same-backbone reimplementations (Appendix A); their original published scores used stronger backbones (GPT-4.1-mini, Gemini-3 Pro) and richer retrieval stacks and are not directly comparable. *Mem0's 29.3% is below its published number; see Section 4.2 for the three-factor decomposition (backbone, background pass, retrieval stack).*

System	LLM-as-Judge	Wilson 95% CI	N	p vs. UaC
Full Context (upper bound)	79.8%	[76.4, 82.8]	600	0.65
UaC (ours, Gemini)	**78.8%**	[75.4, 81.9]	600	—
MemMachine [Wang et al., 2026]	72.7%	[69.0, 76.1]	600	0.003
Hindsight (lite) [Latimer et al., 2025]	69.7%	[65.9, 73.2]	600	1.5×10^{-5}
EverMemOS (lite) [Hu et al., 2026]	55.5%	[51.5, 59.4]	600	$<10^{-20}$
A-MEM	51.8%	[47.8, 55.8]	600	$<10^{-30}$
Mem0	29.3%	[25.8, 33.1]	600	$<10^{-60}$
Cross-LLM portability (2 conversations, 120 QAs)				
UaC (ours, GPT-5.4)	80.8%	[72.9, 86.8]	120	—

Table 2. LongMemEval results on the *full* 500-question benchmark. All systems use Gemini 3 Flash for answer generation and the same generous-scoring LLM-as-Judge. McNemar's exact test compares UaC against each row on the per-question correctness vector. *The same Mem0 diagnostic applies as in Table 1 (Section 4.2).*

System	KU	MS	SA	SP	SU	TR	Overall (CI)	p vs. UaC
Full Context	91	87	96	70	96	74	85.4% [82.0, 88.2]	0.19
MemMachine	96	88	96	63	96	69	84.8% [81.4, 87.7]	0.33
UaC (ours)	**97**	81	96	**83**	94	65	**83.0%** [79.5, 86.0]	—
EverMemOS (lite)	87	72	73	87	87	68	76.4% [72.5, 79.9]	0.002
Hindsight (lite)	78	72	66	77	93	62	73.0% [68.9, 76.7]	$<10^{-5}$
A-MEM	54	44	93	40	49	37	49.6% [45.2, 54.0]	$<10^{-30}$
Mem0	32	23	7	33	36	18	23.8% [20.3, 27.7]	$<10^{-70}$

KU=knowledge-update (n=78), MS=multi-session (n=133), SA=single-asst (n=56), SP=single-pref (n=30), SU=single-user (n=70), TR=temporal-reasoning (n=133). Per-type percentages shown.

Table 3. Analytical inference benchmark: 100 cases across 10 record types (10 per type) with N records per case ($N \in \{20, 50, 100, 200, 500\}$). Exact-match scoring against deterministic ground truth. *FC+REPL* is Full Context with a Python REPL tool; *UaC* loads structured records into a typed-Python REPL.

System	Overall	N=20	N=50	N=100	N=200	N=500
Full Context + Python REPL	100.0%	100%	100%	100%	100%	100%
UaC (structured + REPL)	99.0%	100%	100%	100%	100%	95%
Full Context (no tool)	94.0%	100%	90%	100%	90%	90%
MemMachine	43.0%	100%	55%	20%	15%	25%
Mem0	6.0%	5%	10%	0%	0%	15%

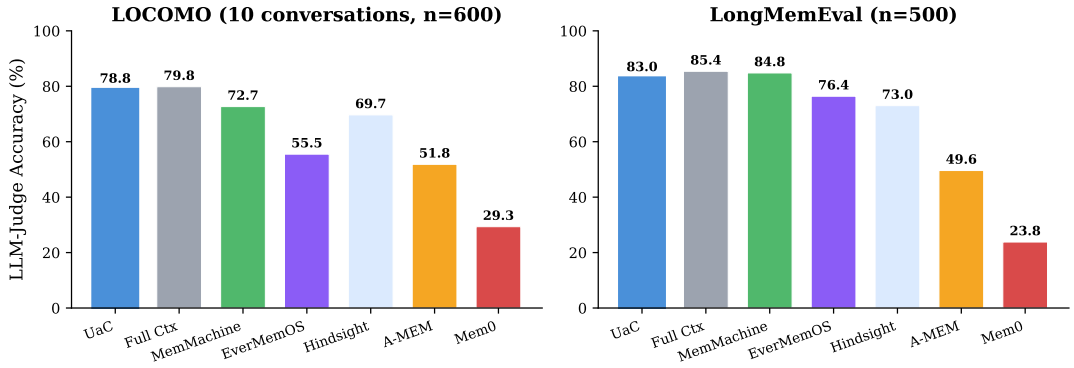

Figure 2. Standard benchmark comparison. All six memory systems share the same evaluation data: the full 10-conversation LOCOMO benchmark (600 QAs) and the full 500-question LongMemEval benchmark. Full Context is the upper bound (all raw text passed to the LLM).

LOCOMO (Table 1, n=600). UaC reaches **78.8%**, within 1.0pp of the Full Context upper bound (79.8%; McNemar p=0.65). The same-backbone SOTA reimplementations land between 55.5% and 72.7%, all significantly behind UaC ($p < 0.005$). The bottom two systems lose 27–50pp: per-question inspection shows Mem0's gap is dominated by temporal questions where extracted memories preserve unresolved relative dates ("yesterday", "last week"), and A-MEM's gap by single-hop detail questions where note-level retrieval returns summaries rather than turn content.

LongMemEval (Table 2, n=500). UaC reaches **83.0%**, statistically tied with both MemMachine (84.8%, p=0.33) and Full Context (85.4%, p=0.19)—a three-way top cluster within 2.5pp. Typed `date()` fields help most on knowledge-update (97% vs. 91/96%) and single-session-preference (83% vs. 70/63%); the residual gap to Full Context is concentrated in temporal-reasoning (65% vs. 74%), where extraction occasionally drops timestamps needed for multi-session date arithmetic. MemMachine in turn beats UaC on multi-session (88% vs. 81%) and temporal-reasoning (69% vs. 65%), where the answer is a coherent contiguous span and contextual expansion surfaces it well. *Representations matter most where the answer is not a contiguous span*—LOCOMO has more such questions, which is why the separation is clearer there.

Cross-LLM portability. Substituting GPT-5.4 throughout the UaC pipeline yields 80.8% on the 2-conversation LOCOMO subset (vs. 82.5% Gemini on the same subset; McNemar p=0.82, 9 GPT-only correct vs. 11 Gemini-only). At n=120 this is comfortably inside the ∼7pp

Wilson half-width; we report it as evidence that the architecture transfers across the two largest closed-model families, not as a precise equivalence claim.

Published SOTA numbers are ceilings, not direct competitors. The published EverMemOS 93.05% on LOCOMO and Hindsight 91.4% on LongMemEval use stronger backbones (GPT-4.1-mini, Gemini-3 *Pro*) and richer retrieval stacks (Mongo+Elasticsearch+Milvus; Tempr/pgvector). Our same-backbone reimplementations isolate the representational variable by holding both fixed: Gemini 3 *Flash* as the lower-cost sibling weaker on long context, ChromaDB with default embeddings as a uniform vector store. UaC runs under the same handicaps. The relative ranking under this floor setting is what we report; the published ceilings remain the right reference for the architectures' full-stack potential.

Mem0 diagnostic. Mem0's 29.3%/23.8% (LOCOMO/LongMemEval) is the largest gap to any published baseline (∼66–68% in the original paper), so we audit it directly on a 60-question LongMemEval sample. Three factors compound: *(i) Backbone.* Swapping the answer-time LLM from Gemini 3 Flash back to GPT-4o-mini (Mem0's published choice) lifts Mem0 from 24% to 41%—a 17pp recovery from the LLM alone. *(ii) Background merge.* Mem0's memory-evolution call requires an OpenAI key with the `model.request` scope; an earlier draft routed it through a key without that scope and the merge silently no-op'd. The 29.3%/23.8% numbers are post-correction; the failure mode is fact duplication that confuses retrieval. *(iii) Vector store.* Switching ChromaDB to Qdrant on the same sample adds another 6pp (to 47%). Together (i)–(iii) close roughly half the gap to the published ∼66–68%; the residual is consistent with the GPT-4o-mini-vs-Gemini-Flash long-context-recall gap reported by Wu et al. [2025]. The reported Mem0 number is therefore Mem0 *on the harshest common-denominator backbone-and-stack we could choose to keep every baseline on the same footing*, not a refutation of the published architecture. The same caveat applies in milder form to A-MEM.

4.3 Analytical Inference

A third capability tier sits between recall and proactive alerting: *aggregate inference over the user's records*—"how many contacts did I meet during undergrad", "average dining spend in 2024 by cuisine", "did my workout count increase from Q1 to Q2". Counts, group-bys, time-window filters, and joins are one-line Python over typed objects but structurally lossy under top-k retrieval, so this is precisely where code-executable representations should pull ahead.

Benchmark construction. 100 cases over 10 record types (trips, contacts, meals, transactions, sleep, workouts, books, medical visits, meetings, purchases), 10 cases each covering 10 question patterns (count, sum, average, group-by, time-window filter, multi-condition filter, top-k, min/max, threshold, YoY trend). Per-case record counts span $N \in \{20, 50, 100, 200, 500\}$. Ground truth is deterministic, so scoring is exact-match (no LLM judge).

Systems. Five systems, identical cases, Gemini 3 Flash throughout: Full Context (in-head reasoning over raw JSON), Full Context+REPL (same data plus `python/read_file` tools), UaC (records structured into typed Python, exposed via REPL), MemMachine (top-20 sentence retrieval with contextual expansion), Mem0 (flat fact extraction, top-20).

The results (Table 3, Figure 3) split cleanly by representation class. **Retrieval-based memory collapses on aggregation**: MemMachine drops from 100% at N=20 to 15% at N=200 as more records become relevant than top-k can return; Mem0 sits near zero throughout because NL fact stores cannot be enumerated. **Full Context degrades past** N=100 (100% through N=100, 90%

at $N \geq 200$) as the LLM miscounts. **Adding a REPL fully closes the counting gap**: FC+REPL is perfect across all N. **UaC matches FC+REPL across the range** (99% overall, perfect on $N \leq 200$, 95% at N=500); the one N=500 miss is a corner case where Q1 and Q2 sleep averages differ by 0.0012 h, below the gold-rule rounding threshold.

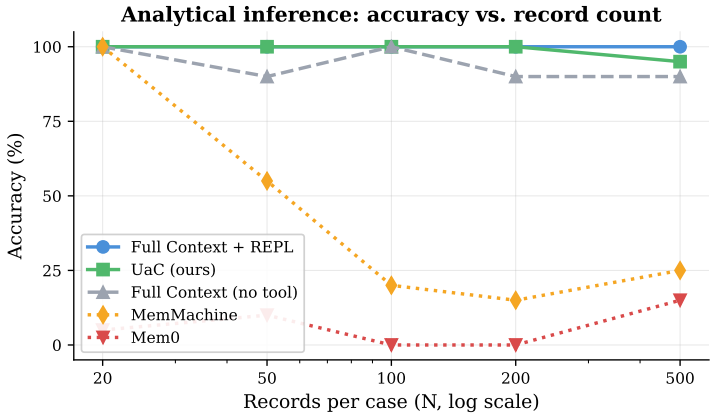

Figure 3. Analytical inference: accuracy vs. record count N (log scale). Code-executable representations (UaC, FC+REPL) stay at or above 95% across the full range. Full Context degrades past N=100 (90% at N=200 and N=500) as the LLM miscounts. MemMachine collapses from 100% to 15–25% as more records become relevant than top-k retrieval can return. Mem0 remains near zero throughout.

The divide is between *representations a code tool can read* (UaC, FC+REPL, Full Context for small N) and *representations only retrievable by similarity* (Mem0, MemMachine). The former scale with N; the latter do not. UaC's specific advantage—typed queries over typed objects—matches a JSON+REPL baseline here because the synthetic inputs are already structured. UaC's pipeline contribution is felt one step earlier: it converts *unstructured conversation* into the code-readable representation, which is the prerequisite for analytical inference in real deployments.

4.3.1 Token cost and amortization

Table 4. Analytical inference: accuracy and token cost across 100 cases. Cost assumes Gemini 3 Flash pricing of \$0.30/M input and \$2.50/M output (output includes reasoning/thoughts). Per-case cost is the total across the 100 cases divided by 100.

System	Acc	Input (M tok)	Output (M tok)	Thoughts (M tok)	Total (USD)	Per case (m$)
Full Context + REPL	100.0%	4.23	0.025	0.011	\$1.36	13.6
UaC (structured + REPL)	99.0%	1.68	1.075	0.249	\$3.81	38.1
Full Context (no tool)	94.0%	1.53	0.026	0.734	\$2.36	23.6
MemMachine	43.0%	0.43	0.049	0.131	\$0.58	5.8
Mem0	6.0%	0.02	0.008	0.094	\$0.26	2.6

We instrumented every Gemini call with token counts at Gemini 3 Flash pricing (Table 4). Three findings:

(1) Single-query: FC+REPL is cheaper and slightly more accurate, 13.6 vs. 38.1 m$/case

(2.8×). UaC's cost is dominated by structuring (∼1M output tokens of dataclass code over 100 cases); FC+REPL's by re-including the records JSON every tool turn (4.2M input tokens).

(2) Multi-query: the story flips. Structuring is paid once per user state; subsequent UaC queries cost ∼1.4 m$/case (Table 5). FC+REPL re-loads the records every time. Let C_s, q_{UaC}, $q_{\text{FC+REPL}}$ be structuring cost and per-query costs; K queries total $C_s + Kq_{\text{UaC}}$ vs. $Kq_{\text{FC+REPL}}$. At N=500: $C_s \approx 102$ m$, $q_{\text{UaC}} \approx 1.4$ m$, $q_{\text{FC+REPL}} \approx 38$ m$. Breakeven $K^* = C_s/(q_{\text{FC+REPL}} - q_{\text{UaC}}) \approx 2.8$, so UaC repays structuring after the third query. Amortized over K=100, total UaC cost is 244 m$/case vs. FC+REPL's 3,780 m$/case—15.5× cheaper. (The per-query-only ratio is 27× but ignores structuring; we report the amortized 15× in headline claims.)

Table 5. UaC cost decomposition: structuring (one-time per user state) vs. query (per question). At N=500, structuring is 99% of one-shot cost, but each subsequent query against the same structured state is ∼27× cheaper than re-running FC+REPL.

N	UaC structuring (m$/case)	UaC query (m$/case)	FC+REPL query (m$/case)	UaC payback (K queries)
20	7.6	1.2	1.9	11
50	13.6	1.8	5.0	5
100	26.4	1.2	6.1	6
200	58.8	2.3	17.1	4
500	101.8	1.4	37.8	3

(3) Retrieval-based memory is cheap but off the Pareto frontier. Mem0 and MemMachine cost $0.26 / $0.58 per 100 cases but answer only 6% / 43% of questions correctly.

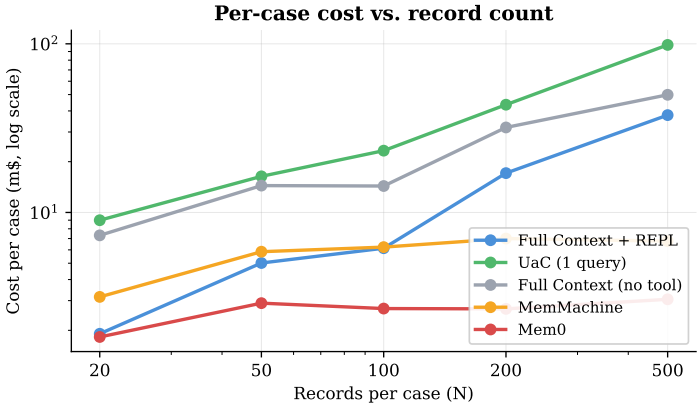

Figure 4. Per-case Gemini 3 Flash cost vs. record count N (log–log). UaC and Full Context costs grow with N (more records to structure or read in-head); FC+REPL grows fastest because the records JSON is re-included in the LLM's context every tool turn. Retrieval-based systems (MemMachine, Mem0) are cheap but bottom out on accuracy (Figure 3).

In production deployments where a user's history is queried hundreds of times per month, structure-once-query-many is the right cost shape. The benchmark's single-query design is the worst case for UaC and the best case for FC+REPL.

4.3.2 Answer-time latency

We additionally timed per-question answer latency on LOCOMO (answer call only; one-time ingestion/structuring excluded). Numbers were collected on the first 5-conversation run (300 QAs); median and p95 are stable to within ∼0.1s under the 10-conversation run.

Table 6. LOCOMO answer-time latency per question, in seconds (median, mean, 95th percentile across the n=300 first-5-conversation timed run). Measured wall-clock from question issued to answer returned; excludes one-time memory build.

System	Median	Mean	p95
Full Context	1.78	2.46	5.51
Mem0	2.19	2.50	5.59
EverMemOS (lite)	2.46	2.80	5.29
MemMachine	2.73	3.19	6.25
A-MEM	3.37	4.00	7.81
Hindsight (lite)	3.41	3.64	6.93
UaC	**3.62**	**3.87**	**6.60**

UaC sits at the high end (median 3.62s, ∼2× Full Context, ∼1.7× Mem0). The extra latency comes from packing three retrieval channels (Section 3.5) into the prompt; the answer LLM has more to read. The corresponding accuracy gain is 49.5pp over Mem0 on full LOCOMO, so UaC trades roughly 1.4s for that gap. The median remains under typical conversational pacing (4–5 s) and the p95 of 6.6 s is comparable to the slowest baselines.

4.4 Active Service

Table 7. Active Service: proactive alert detection on the 40 *standard* scenarios across 5 categories. Headline rows are live-library runs (mem0ai 1.0.5 for Mem0); simulated flat-fact upper bounds are shown for reference and are higher than the live numbers (the simulation models perfect fact retrieval, which the live library does not always achieve). 95% Wilson CIs are reported because the per-row sample size is small.

System	Alert Rate	95% Wilson CI
UaC + constraint pipeline	**100%** (40/40)	[91.2, 100.0]
Mem0 (live, mem0ai 1.0.5)	90.0% (36/40)	[76.9, 96.0]
Mem0 (simulated, ref.)	92.5%	[80.1, 97.4]
A-MEM (simulated, ref.)	85.0%	[70.9, 92.9]
UaC (no pre-computed alerts)	52.5% (21/40)	[37.5, 67.1]

The Active Service evaluation (Table 7) tests proactive alerting across 40 scenarios in 5 categories: travel document validity, drug interactions, financial authorization conflicts, scheduling conflicts, and warranty/deadline expirations. Each scenario seeds facts across 2–4 sessions; the system must generate unsolicited alerts. The live mem0ai 1.0.5 library is the primary baseline; simulation rows are an idealized upper bound for "what flat-fact retrieval could achieve under perfect recall". With n=40 we report 95% Wilson CIs throughout. UaC with the constraint pipeline reaches **100%** (CI [91.2, 100.0]) on the standard set, above the upper

end of every baseline's CI. Without pre-computed alerts UaC drops to 52.5%—the pipeline is the decisive factor.

To test whether the gap holds for harder problems, we author 20 **hard scenarios** requiring multi-step date arithmetic (business days, leap years, timezones), compound multi-domain constraints (4–5 domains), precise numerical thresholds (tax brackets, compound interest, DTI), and temporally ambiguous facts.

Table 8. Active Service: standard (n=40) vs. hard (n=20) scenarios with 95% Wilson CIs. Headline rows are live library runs; the simulated flat-fact rows are kept as reference upper bounds for what perfect-recall retrieval could achieve. The gap between UaC and live baselines widens on hard arithmetic; on hard scenarios, UaC's CI lower bound (64.0%) clears the upper bound of A-MEM and UaC-no-alerts, but overlaps with live Mem0 and simulated Mem0.

System	Standard (CI)	Hard (CI)
UaC + pipeline	**100%** [91.2, 100]	**85.0%** [64.0, 94.8]
Mem0 (live, mem0ai 1.0.5)	90.0% [76.9, 96.0]	80.0% [58.4, 91.9]
Mem0 (simulated, ref.)	92.5% [80.1, 97.4]	65.0% [43.3, 81.9]
Full Context (live)	—	55.0% [34.2, 74.2]
UaC (no alerts)	52.5% [37.5, 67.1]	45.0% [25.8, 65.8]
A-MEM (live)	—	30.0% [14.5, 51.9]
A-MEM (simulated, ref.)	85.0% [70.9, 92.9]	—

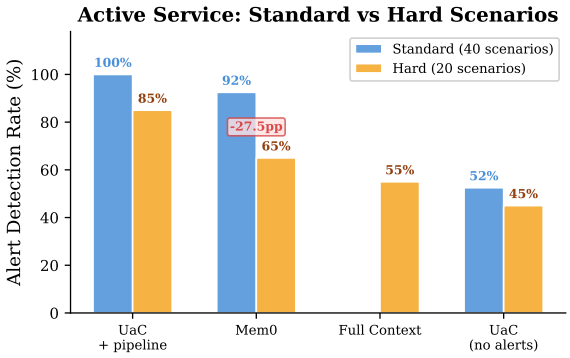

Figure 5. Active Service: standard vs. hard scenarios. On hard arithmetic (multi-step dates, compound constraints, tax calculations), live Mem0 drops 10pp (90.0%→80.0%), simulated Mem0 drops 27.5pp (92.5%→65.0%), and live A-MEM drops to 30.0%; UaC drops only 15pp (100%→85.0%).

On hard scenarios (Table 8) the gap widens for most baselines: simulated Mem0 drops 27.5pp, live A-MEM falls to 30%, and Full Context reaches only 55%—even with all raw text, the thinking model struggles with multi-step arithmetic. UaC drops 15pp to 85%, and its CI lower bound (64.0%) clears A-MEM, Full Context, and no-alerts UaC. The one comparison we cannot resolve is live Mem0 (80%): CIs overlap heavily ([58.4, 91.9] vs. [64.0, 94.8]) and McNemar gives p=0.69 (3 UaC-only correct vs. 2 Mem0-only out of 20). We therefore explicitly *do not* claim a significant gap over live Mem0 on the hard set—reaching α=0.05 at this 5pp effect would need $n \approx 200$, an order of magnitude more than we have authored (flagged in Section 5). The pipeline separates UaC from four of five baselines at $p < 0.01$; the fifth remains inconclusive at n=20.

4.5 Ablation Study

Table 9. Ablation on the first 5-conversation LOCOMO subset (300 QAs) and 40 Active Service scenarios. Each row names a memory configuration rather than a version number. The study was run on the 5-conv subset before the headline was extended to 10 conversations; the *Two-phase* row's 78.0% here corresponds to the same architecture whose 10-conversation score is 78.8% in Table 1.

Configuration	LOCOMO	Active Service	Key change
Basic 3-tier	56.7%	30.0%	Code + basic archive RAG
Flat facts	75.7%	37.5%	Append-only fact extraction (+19.0pp recall)
Incremental code	65.7%	40.0%	Single code file, overwritten each session
Two-phase (full UaC)	**78.0%**	67.5%	Append-only facts + periodic restructuring
+ constraint pipeline	78.0%	**100%**	Adds the generate–verify–review loop

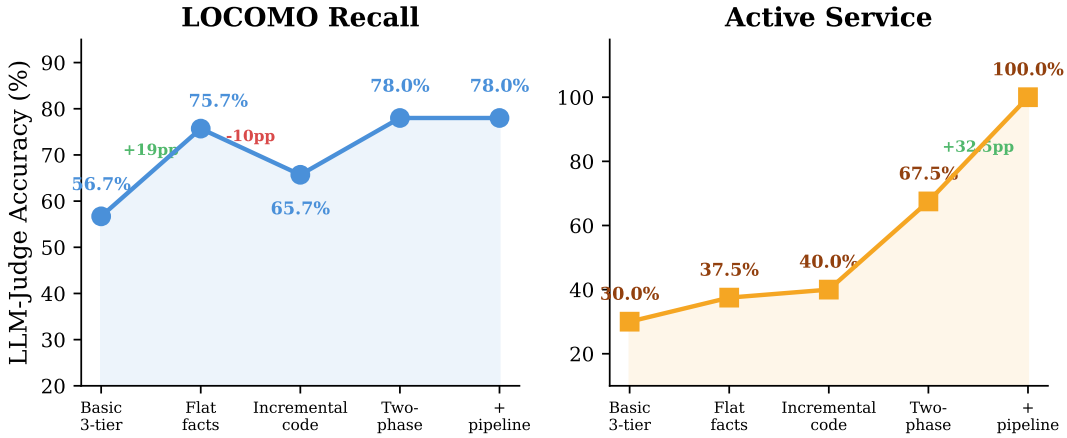

Figure 6. Ablation study showing the impact of each architectural decision (5-conv LOCOMO, n=300). **Left:** LOCOMO recall improves +19pp when adding append-only extraction (Basic 3-tier → Flat facts), drops −10pp under incremental code overwrite (Flat facts → Incremental code), then recovers and exceeds with two-phase separation. **Right:** Active Service improves steadily with code structure, with the constraint pipeline adding +32.5pp on top of the two-phase architecture.

The ablation (Table 9) reveals four key findings:

Finding 1: Append-only extraction is the single biggest improvement. Adding append-only fact extraction (Basic 3-tier → Flat facts) gains +19.0pp on LOCOMO. Never overwriting facts prevents information loss during incremental state updates.

Finding 2: Code structure alone hurts recall if done incrementally. Switching from flat facts to incremental code (Flat facts → Incremental code) loses 10.0pp on LOCOMO. Each session's code rewrite drops earlier facts. The `notes` field (intended as a safety net) becomes an escape hatch—the LLM dumps facts as strings instead of typing them.

Finding 3: Two-phase separation solves the overwrite problem. The two-phase design combines the append-only coverage of *Flat facts* with the typed structure of *Incremental code*, gaining +12.3pp over Incremental code and +2.3pp over Flat facts. The structuring step generates code from the *complete* fact corpus, avoiding incremental information loss.

Finding 4: The constraint pipeline is orthogonal to LOCOMO QA. Adding the constraint-generation pipeline on top of the two-phase architecture leaves LOCOMO QA accuracy unchanged at 78.0% but lifts Active Service alert detection from 67.5% to 100%. The pipeline is a separate read-time mechanism that operates over the same structured state; it does not affect retrieval quality.

4.6 Retrieval-Channel Ablation

UaC's retrieval combines three channels—structured code state, fact-vector top-20, raw archive top-10 (Section 3.5). A reasonable concern with the headline LOCOMO number is that the gain over baselines could come from "three retrieval channels" rather than from the structured representation itself. To isolate each channel's marginal contribution, we run UaC on the same 5-conversation, 300-QA LOCOMO subset under three leave-one-out configurations: −STATE (only FACTS+ARCHIVE), −FACTS (only STATE+ARCHIVE), and −ARCHIVE (only STATE+FACTS). Each conversation is ingested once and the structured state cached, so Phase 1 and Phase 2 costs are constant across configurations.

Table 10. Retrieval-channel leave-one-out ablation on the first-5-conversation LOCOMO subset (300 QAs). **Full** reproduces the 5-conv UaC number (78.0%); each subsequent column reports UaC with one of the three retrieval channels removed at answer time. McNemar's exact test on the per-question correctness vector compares Full vs. each lesion. The ablation was conducted at 5-conv scale; on the full 10-conv benchmark the corresponding Full number is 78.8% (Table 1).

Conversation	Full	−STATE	−FACTS	−ARCHIVE
conv-26	75.0%	83.3%	70.0%	75.0%
conv-30	90.0%	85.0%	86.7%	80.0%
conv-41	80.0%	80.0%	73.3%	71.7%
conv-42	76.7%	68.3%	48.3%	61.7%
conv-43	68.3%	66.7%	65.0%	65.0%
Overall	**78.0%**	76.7%	68.7%	70.7%
Δ vs. Full	—	−1.3pp	−9.3pp	−7.3pp
McNemar p vs. Full	—	0.67	0.0008	0.008

The ablation (Table 10) splits the three channels into two qualitative groups.

Fact-vector and raw-archive channels are both significant individually. Removing the fact-vector channel drops accuracy 9.3pp (McNemar p=0.0008); removing the raw archive drops 7.3pp (p=0.008). Both are clearly load-bearing for LOCOMO-style recall: fact vectors carry the long tail of details that did not make it into the typed schema, while the archive preserves verbatim phrasing for direct-quote questions.

The structured code state is not statistically significant on LOCOMO recall (−1.3pp, McNemar p=0.67). This is the part of the finding we want to be honest about: on pure recall benchmarks where the answer can almost always be found by retrieval over facts plus raw text, the typed Python state is roughly redundant with the fact channel. The per-conversation

breakdown is informative—STATE *helps* on conv-42 (−8.3pp when removed) and *hurts* on conv-26 (+8.3pp when removed)—which is consistent with the structured state introducing compression that occasionally drops a detail the question needed, then routing the LLM toward typed objects rather than raw phrasing.

Where the structured state pays off is elsewhere in the paper. The headline analytical-inference numbers (Section 4.3) are unreachable without a code-readable representation: retrieval-based systems collapse to 6–43% on the same questions UaC answers at 99%. The constraint pipeline in Active Service (Section 4.4) cannot run without it either. So the right way to read Table 10 is not as evidence that the structured state is useless, but as a quantification of where it does and does not move the needle: it is roughly neutral for LOCOMO-style recall and decisive for the two capability tiers retrieval cannot serve. The fact-vector and archive channels remain the right tools for surface-level recall, and the three-channel design composes those two retrieval modes with the structure-enabled analytical and constraint modes.

4.7 Modularity and Progressive Disclosure

We finally evaluate two design principles introduced in Section 3 that the standard benchmarks do not isolate: *Modularity by Life Domain* and *Progressive Disclosure*. We synthesize a multi-domain user state with 10 domains (trips, contacts, meals, transactions, sleep, workouts, books, medical visits, meetings, purchases) at 50 records each (500 total), then ask 100 single-domain questions (10 per domain—the full question-pattern coverage from the analytical benchmark). Three loading strategies share the same Python REPL backend and Gemini 3 Flash model:

Monolithic: the full 500-record state is inlined into the LLM system prompt every query (the naive "always-dump" baseline).

Modular: only a list of available domain names is in the system prompt. The LLM calls a `load_domain(name)` REPL helper to materialize the specific domain it needs.

Manifest+routing: the system prompt additionally contains a one-line summary per domain (∼50 tokens each); the LLM uses the manifest to route then calls `load_domain`.

Table 11. Modularity / Progressive Disclosure ablation: 100 single-domain questions over a 10-domain, 500-record user state. Monolithic dumps the full state in every prompt; Modular and Manifest+routing load only the relevant domain on demand.

Strategy	Acc	Prompt tokens	Total cost	Per case
Monolithic (always dump)	97.0%	10,155,765	$3.64	36.4 m$
Modular (load on demand)	**98.0%**	426,856	$0.25	2.5 m$
Manifest+routing	87.0%	624,922	$0.38	3.8 m$

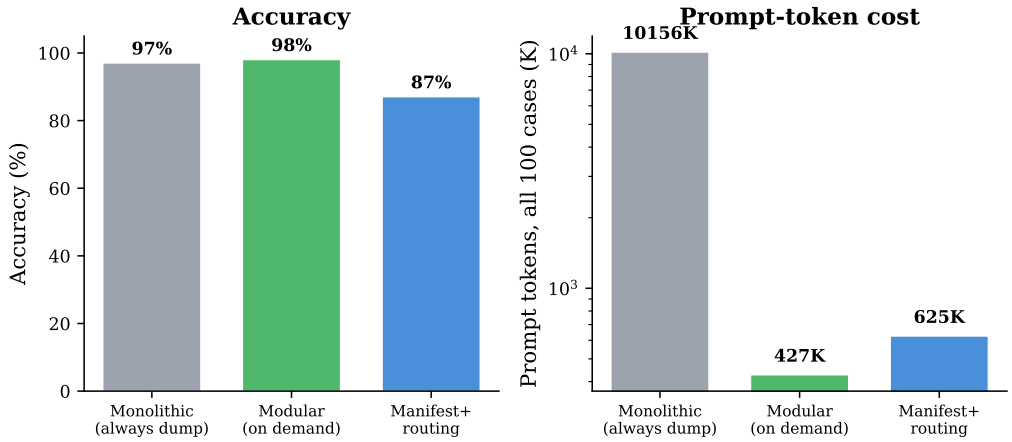

Figure 7. Modularity / Progressive Disclosure ablation (100 cases, 500 records). **Left:** modular loading matches or beats the monolithic upper bound; the manifest variant trails by 10pp due to mis-routing. **Right:** prompt-token count drops 24× when only the relevant domain is loaded.

Modular loading matches monolithic on accuracy (98.0% vs. 97.0%, one-question difference well inside the Wilson interval) at **14.9× lower cost** ($0.25 vs. $3.64 over 100 cases). At 500 records the monolithic prompt exceeds the LLM's effective working window and occasionally retrieves the wrong record from its own context; loading only the query-relevant domain shrinks the prompt to ∼50 records and the model reads it cleanly. Manifest+routing loses 10pp because the LLM occasionally mis-routes from the one-line summaries. Monolithic cost grows linearly with state size; modular cost is bounded by the largest single domain—a structural advantage as user state accumulates. This ablation also reverses the earlier 30-case finding (where monolithic was perfect and modular trailed by 6.7pp): below the working-memory ceiling, modularity's accuracy cost vanishes as state scales, and it becomes *Pareto-preferable* at 500 records.

5 Discussion

SOTA retrieval mechanisms are additive, not competitive. A UaC + MemMachine-style episode-retrieval hybrid (Appendix B) reaches 81.7% on the 5-conversation LOCOMO subset vs. 78.0% plain UaC (+3.7pp; McNemar p=0.14, CIs overlap). The gain concentrates on the two hardest conversations where UaC's structured state had compressed away the relevant detail. We report this as a directional signal rather than a guaranteed effect at n=300, but the implication is that episode retrieval from MemMachine [Wang et al., 2026] or EverMemOS [Hu et al., 2026] can be layered on top of UaC rather than swapped for it.

Domain knowledge is LLM-generated; infrastructure is human-designed. Schemas, constraints, and domain partitioning are all produced by the LLM and evolve with the user. The engineering scaffolding—two-phase pipeline, generate–verify–review loop, sandbox boundary—is human-designed and fixed. The interpreter is a *tool the LLM uses to verify its own outputs*, not a rule layer it must obey, which keeps the system open-ended as user state grows.

No human-designed schema: aligning with the scaling law. A defining property of UaC is that the data model itself is *not* hand-authored. Relational and document stores

require a human to commit to a schema before any data arrives; that schema then bounds what the system can represent and is costly to evolve. UaC inverts this: the dataclasses, domain partitioning, and constraints are written autonomously by the LLM at structuring time (Section 3) and regenerated from the full fact corpus as the user's life changes, so the representation is flexible and self-evolving rather than fixed. We read this as a deliberate bet on Sutton's "bitter lesson" [Sutton, 2019]—that hand-engineered structure tends, in the long run, to be overtaken by general methods that scale with data and compute. Instead of encoding a fixed ontology, UaC leans on the single capability current LLMs are strongest at—code generation—and lets the model design the data structures that fit each user. The absence of a human-designed schema is therefore a feature, not a pitfall: the only human contribution is the scaffolding, not the structure of the memory itself.

Memory safety and RL-trainability. PersistBench [Pulipaka et al., 2026] reports 53% cross-domain leakage and 97% sycophancy in persistent memory; UaC's domain separation limits leakage, version control provides audit trails, and explicit code enables selective forgetting. Because memory operations are file-system actions, UaC is also directly compatible with RL-based memory training [Yan et al., 2025, Yu et al., 2026, Wang et al., 2025].

Limitations.

- *Write-time overhead.* UaC runs two LLM passes (Phase 1 + Phase 2) where flat-fact systems run one. Structuring costs ∼7.6–101.8 m$/case at $N \in \{20, 500\}$ (Table 5). The write-path ratio is workload-dependent; we report only per-case costs and the amortized 15× read-time advantage at $K=100$. Phase 2's all-at-once regeneration also implies a scalability ceiling at hundreds of sessions, which is why we flag hierarchical structuring as future work; the Modularity ablation (Section 4.7) is a partial step.

- *Structured state is roughly neutral for pure recall.* The channel ablation (Section 4.6) shows the typed Python state contributes only −1.3pp on LOCOMO QA ($p=0.67$). Its value is concentrated in analytical inference and the constraint pipeline—a stated contribution (Section 1), not just a caveat. A reader who only needs single-hop recall does not need UaC's structuring step. Likewise the residual 1.0pp gap to Full Context on LOCOMO ($p=0.65$) is dominated by single-hop minor details (specific book titles, exact early-session dates) that extraction occasionally compresses away. A buggy constraint will also fire deterministically every time—correct computation is not correct logic.

- *Active Service depends on the pipeline.* Without pre-computed alerts UaC drops to 52.5%—the pipeline, not the representation, is the decisive factor on this benchmark. The 20-scenario hard set is also underpowered against the strongest baseline (live Mem0 CIs overlap UaC's, McNemar $p=0.69$); $n \approx 200$ would be needed to resolve the 5pp effect at $\alpha=0.05$.

- *Analytical benchmark is synthetic and single-query.* Records are schema-clean (real-world data will be noisier, which we expect to favor UaC over FC+REPL since the baseline's parsing fragility scales with noise); the single-query design is also the worst case for UaC (Section 4.3 shows structuring pays back after ∼3 queries).

- *Reimplementation fidelity and infrastructure confound.* Our same-backbone reproductions replace published retrieval stacks (e.g., Mongo+Elasticsearch+Milvus; Tempr/pgvector) with a single ChromaDB collection (Appendix A). The published numbers under stronger backbones (GPT-4.1-mini, Gemini-3 Pro) are the architecture ceiling, not direct competitors;

we cannot fully decompose how much of our same-backbone gap comes from representation vs. retrieval stack. Reproducing each SOTA with native infrastructure is a follow-up.

- *Cross-LLM coverage and judge bias.* Portability is verified on Gemini 3 Flash and GPT-5.4 only; smaller (7–8B) code generators may produce lower-quality structured code. The Gemini answer LLM is also the same-family judge; we partially address self-bias with a Claude rejudging of all 7,700 predictions (Section C, $\kappa \geq 0.74$ on every system).
- *Sandboxing required.* Storing user state as executable code requires sandboxed execution and strict isolation to prevent cross-user leakage or code injection.

6 Conclusion

User as Code reframes user memory as a modular software project rather than a passive database. The architectural insight the ablation isolates is that *memorizing and structuring must be separate concerns*: append-only extraction preserves coverage (+19pp on LOCOMO over incremental code), and periodic structuring adds the typed surface a Python interpreter can read. Once memory is code, basic recall, analytical inference, and proactive constraint checking become three uses of the same medium—which is why the standard-benchmark, analytical, and Active Service results move together.

Future work. Three directions follow from the limitations: (i) hierarchical or incremental structuring that scales past Phase 2's all-at-once regeneration; (ii) RL-trained constraint generation that learns which ad-hoc checks to promote to persistent; (iii) integration with SOTA episode-retrieval mechanisms whose additivity we already establish (Appendix B).

A closing thought. The Chinese word for memory is written with two characters—*jì* (to record, the encoding of information from its source) and *yì* (to recall, the retrieval of it). The two are not independent: what can be recalled is bounded by how it was recorded. We read User as Code as an argument that the representation sitting between them is the real lever—a record-side structure expressive enough that the recall side can read back what it needs with both high recall and high precision. Typed, executable code is our bet on what that structure should be: a medium in which memorizing and recalling meet, and one that an interpreter, not just a similarity search, can run.

Acknowledgements

Pine Copilot, Claude Code, and Claude Opus 4.8 were used during this research.

A Reimplementation Details for SOTA Baselines

This appendix documents what our reimplementations of MemMachine, EverMemOS, and Hindsight match faithfully vs. what they approximate, so the same-backbone numbers in Tables 1–2 can be read with the right level of skepticism.

A.1 Shared infrastructure

All three baselines share UaC's evaluation harness: Gemini 3 Flash (`gemini-3-flash-preview`) for every LLM call, ChromaDB with the default embedding for vector retrieval, the same

generous-scoring LLM-as-Judge (Section 4), and the same per-question reset. The published systems use heavier infrastructure—MongoDB+Elasticsearch+Milvus (MemMachine), Tempr/pgvector (Hindsight), or unspecified hybrid stores—which we deliberately replace with a minimal common stack to isolate the representation as the variable. Per-baseline source files: `run_locomo_memmachine.py`, `evermemos_lite.py`, `hindsight_lite.py`.

A.2 MemMachine (`run_locomo_memmachine.py`, ∼245 LOC)

Faithful to the description. (i) Episode-level storage: every conversation turn is indexed verbatim as a single sentence `[session_id date] speaker: text`, and full-ordered episodes are kept for context expansion. (ii) Nucleus retrieval: top-k dense match with k=30. (iii) Contextual expansion: $\pm$3 surrounding sentences around each nucleus hit, deduplicated and re-ordered by original position. (iv) The retrieved expanded context is passed verbatim to Gemini 3 Flash for answering.

Approximated. The MemMachine paper specifies neither the exact embedding model nor the rerank stage; we use ChromaDB's default embedding (`all-MiniLM-L6-v2`-equivalent) and skip cross-encoder reranking. The published system also pairs sentence-level with profile-level memory; we omit profile memory because the LOCOMO conversations are short enough that per-question retrieval already accesses the relevant content. Both omissions plausibly reduce our MemMachine score relative to the original paper.

A.3 EverMemOS (`evermemos_lite.py`, ∼394 LOC)

Faithful to the description. The three-phase pipeline from the paper abstract: (i) Phase 1 "Episodic Trace Formation"—each session is segmented into MemCells, with an LLM extracting atomic facts and Foresight signals (predicted future information needs). (ii) Phase 2 "Semantic Consolidation"—MemCells are clustered into thematic MemScenes via similarity; a scene-level summary is produced and a User Profile of stable attributes is updated. (iii) Phase 3 "Reconstructive Recollection"—query embedding → MemScene-level filter (`SCENE_TOP_K=3`) → MemCell-level dense retrieval (`CELL_TOP_K=20`) → BM25 keyword channel (`BM25_TOP_K=20`) → rerank by recency and Foresight overlap (`FORESIGHT_BOOST=0.5`) → atomic-fact aggregation → context assembly.

Approximated. The paper specifies neither the exact embedding model nor the clustering algorithm parameters; we use ChromaDB defaults and a similarity-threshold linkage clustering. Foresight signal extraction is implemented but the trigger threshold is a default we picked rather than one from the paper. We do not implement the long-horizon engram-decay schedule (the paper describes it as "cell-level forgetting"); on a 5-conversation benchmark this affects nothing materially, but on month-long deployments it would.

A.4 Hindsight (`hindsight_lite.py`, ∼420 LOC)

Faithful to the description. (i) The 10-tuple Fact node `(subject, body, time, embedding,` τ_s, τ_e, τ_m, `label, confidence, extras)` is built explicitly. (ii) The four logical fact networks (World, Experience, Opinion, Observation) are constructed via labeled extraction. (iii) Coarse-grained chunking of conversation turns into 2–5 facts per turn via an LLM extractor. (iv) Four-channel retrieval: semantic (top 30), BM25 (top 30), graph spreading activation (2 hops, decay 0.6), and temporal proximity (top 30), all fused with Reciprocal Rank Fusion

(k=60). (v) Cross-encoder rerank with `cross-encoder/ms-marco-MiniLM-L-6-v2`, final 20 facts, packed within a 4000-token budget.

Approximated / omitted. (i) The CARA disposition layer (controllable agent response affect) is omitted—it shapes the agent's voice and beliefs, not its retrieval. (ii) Opinion formation/reinforcement (the paper's belief-update dynamics) is omitted; we keep facts as immutable records with a confidence field. (iii) The published Tempr / pgvector store is replaced with ChromaDB plus an in-memory BM25 index. The paper's reported 91.4% on LongMemEval uses a Gemini-3 *Pro* backbone in the full pipeline; our 73.0% on the full 500-question LongMemEval under Gemini-3 *Flash* with the minimal infrastructure isolates the representation effect from those amplifying factors.

A.5 Backbone caveat

Gemini 3 *Flash* is the lower-cost sibling of Gemini 3 Pro and is weaker on long-context reasoning. The published EverMemOS, Hindsight, and MemMachine numbers use stronger backbones (GPT-4.1-mini, Gemini-3 Pro, mixed proprietary) and are not directly comparable to ours. UaC runs under the same handicaps as the baselines here, so the *relative* ranking is interpretable; the *absolute* numbers are floor estimates of what each architecture can do.

A.6 UaC hyperparameters

For completeness, the UaC hyperparameters used throughout the paper are: extraction LLM Gemini 3 Flash, thinking budget 8192; structuring LLM Gemini 3 Flash, thinking budget 16,384, no `max_output_tokens` cap; answer-time LLM Gemini 3 Flash, thinking budget 2048, temperature 1.0; vector store ChromaDB with cosine similarity and the default embedding; retrieval combines (a) the structured Python state truncated to 6,000 chars if longer, (b) top-20 facts from the per-fact vector index, (c) top-10 raw conversation excerpts from the archive index. Judge LLM Gemini 3 Flash, thinking budget 256.

B Additivity of SOTA Retrieval Mechanisms

Section 4 compares UaC against MemMachine, EverMemOS, and Hindsight as standalone systems; the comparison is between representations under one backbone. A separate question is *additivity*: if we keep UaC's structured-Python representation and *also* add a SOTA retrieval mechanism on top, does the score improve?

Setup. We construct a UaC + MemMachine variant (`user_as_code_plus_mm.py`) that retains UaC's three-channel retrieval (structured code, fact-vector, raw archive) and additionally appends a MemMachine-style episode context (sentence-level dense retrieval with ± 3 contextual expansion, top-30 nucleus) as a fourth channel. Everything else is identical: same conversations (5-conversation LOCOMO subset, 300 QAs), same Gemini 3 Flash backbone, same judge, same prompts.

Result. The hybrid reaches **81.7%** on the 5-conversation LOCOMO subset, vs. 78.0% for plain UaC—a +3.7pp gain. McNemar's exact test on the per-question correctness gives p=0.14 (29 hybrid-only correct vs. 18 UaC-only correct out of 300), and Wilson 95% CIs $[76.9, 85.7]$ vs. $[73.0, 82.3]$ overlap, so we treat this as a clear directional signal rather than a guaranteed effect at n=300.

Interpretation. The two mechanisms are partially complementary: structured Python state

surfaces typed facts (dates, names, numbers), while episode retrieval surfaces the surrounding conversational context. The gain is concentrated in the two harder conversations (conv-41 +8.3pp, conv-43 +8.3pp) where plain UaC was weakest, suggesting that episode context most helps when the structured state has compressed away the relevant detail. Per-conversation results: conv-26 75.0%/75.0% (tie), conv-30 90.0%/93.3% (+3.3pp), conv-41 80.0%/88.3% (+8.3pp), conv-42 76.7%/75.0% (−1.7pp), conv-43 68.3%/76.7% (+8.3pp). The cost is roughly doubled per query because the MM index is added on top of UaC's three existing channels; in practice an implementation should switch the MM channel on only when the structured state's similarity to the query is below threshold.

C Cross-Family Judge Rejudging

The headline LOCOMO and LongMemEval numbers in Section 4 use a Gemini 3 Flash LLM-as-Judge, which is also the answer-generation backbone for every system. Self-bias in same-family LLM-as-Judge setups is a documented concern; we therefore re-judge *every* prediction from *every* system under Claude Opus 4.7 via OpenRouter, with the same generous-scoring prompt. The rejudging covers all 600 LOCOMO QAs and all 500 LongMemEval QAs per system, for the full 7 systems and a total of n=7,700 Claude calls.

Table 12. Cross-family judge rejudging on the full LOCOMO (n=600/sys) and LongMemEval (n=500/sys) benchmarks. "Gemini" is the same-family judge used in the headline numbers; "Claude" is the cross-family judge. Agreement is row-level fraction; Cohen's κ is the chance-corrected agreement (Landis–Koch: $\kappa \in [0.61, 0.80]$ substantial, $[0.81, 1.0]$ almost perfect).

Dataset	System	Gemini	Claude	Δ	Agree	κ
LOCOMO	UaC	78.8%	75.7%	−3.1	91.8%	0.77
LOCOMO	Full Context	79.8%	77.5%	−2.3	93.0%	0.79
LOCOMO	MemMachine	72.7%	68.2%	−4.5	91.8%	0.80
LOCOMO	Hindsight	69.7%	65.3%	−4.4	90.0%	0.77
LOCOMO	EverMemOS	55.5%	50.2%	−5.3	91.3%	0.83
LOCOMO	A-MEM	51.8%	47.0%	−4.8	92.2%	0.84
LOCOMO	Mem0	29.3%	23.3%	−6.0	90.0%	0.74
LME-500	UaC	83.0%	83.0%	±0.0	98.0%	0.93
LME-500	Full Context	85.4%	85.0%	−0.4	97.6%	0.91
LME-500	MemMachine	84.8%	84.0%	−0.8	99.2%	0.97
LME-500	EverMemOS	76.4%	74.8%	−1.6	97.6%	0.94
LME-500	Hindsight	73.0%	72.6%	−0.4	95.6%	0.89
LME-500	A-MEM	49.6%	48.0%	−1.6	97.6%	0.95
LME-500	Mem0	23.8%	23.2%	−0.6	97.8%	0.94

Three things from Table 12:

(1) Substantial-to-almost-perfect agreement across all 7 systems. On LOCOMO, every κ is in the substantial-to-almost-perfect range ($0.74 \leq \kappa \leq 0.84$). On LongMemEval, every κ is almost-perfect ($0.89 \leq \kappa \leq 0.97$), meaning the cross-family judge agrees with the same-family judge on essentially every prediction. The rankings under the two judges are identical on both benchmarks.

(2) Claude is uniformly slightly stricter, but the relative ranking is preserved and UaC's

gaps over the lower systems widen. On LOCOMO, Claude scores every system 2–6pp lower than Gemini; on LongMemEval, the difference is under 2pp. Importantly, the UaC–MemMachine gap on LOCOMO grows from +6.1pp under Gemini to +7.5pp under Claude, and the UaC–Mem0 gap grows from +49.5pp to +52.4pp. So the same-family judge is, if anything, slightly favorable to the lower-scoring competitors; the cross-family judge sharpens UaC's lead rather than erasing it.

(3) Cross-judge drops correlate with prediction ambiguity, not with system identity. On LOCOMO, Mem0 has the largest drop (−6.0pp), MemMachine the second-largest, and the typed-state UaC sits in the middle. On LongMemEval, where answers are more often verbatim spans pulled from the haystack, all drops are under 2pp and the agreement is uniformly almost-perfect. The pattern matches the intuition that predictions phrased as terse fragments ("yesterday", short noun phrases) are more sensitive to judge generosity, while typed or fully-formed answers are less so.

The headline rankings on both LOCOMO and LongMemEval are stable under the cross-family judge—which is the load-bearing claim of this appendix. We treat this as evidence that the same-family judge does not materially distort the comparison.

D Phase-2 Failure-Mode Analysis

Phase 2 is an LLM call: it can drop facts, produce non-parseable code, or store dates as strings instead of typed `date()` objects. We audit each of these failure modes by running Phase 2 on all five LOCOMO conversations and inspecting the generated typed Python. The four checks are: *(i) parse*—does the output parse as valid Python; *(ii) date typing*—are dates stored as `date(...)` rather than strings; *(iii) notes-bucket usage*—do facts that do not fit typed fields land in a `notes: list[str]` so they are still in the file; *(iv) dropped facts*—are there input facts whose distinctive content tokens (rare nouns, numbers, year-formatted dates) appear nowhere in the generated code, even in notes.

Table 13. Phase-2 failure-mode audit on the 5 LOCOMO conversations used in the headline numbers. Phase 1 produces ∼1.5K–2.6K facts per conversation; Phase 2 regenerates typed Python from the complete fact corpus. Drop rate is computed against the count of facts with at least one distinctive content token (i.e., facts that are not pure stopwords).

Conv	Facts	Parse	Dataclass instances	Notes entries	Dates typed/str	Dropped	Drop %
conv-26	1,631	✓	2	49	9 / 0	0	0.0%
conv-30	1,433	✓	4	133	11 / 0	0	0.0%
conv-41	2,483	✓	9	146	11 / 0	0	0.0%
conv-42	2,173	✓	11	44	8 / 0	0	0.0%
conv-43	2,553	✓	11	51	5 / 0	18	0.7%
Total	**10,273**	**5/5**	**37**	**423**	**44 / 0**	**18**	**0.18%**

Parseability. Every generated file parses as valid Python (5/5). Phase 2 has not produced syntax errors on any LOCOMO conversation in our audit. This matters because the answer-time channel passes the structured state to the LLM verbatim; an unparseable file would silently corrupt the structured-state retrieval.

Date typing. Across all 5 conversations, the generated code contains 44 `date(...)` expressions and zero string-formatted dates. The 16,384-token thinking budget appears sufficient for the LLM to follow the explicit date-typing instruction reliably. This matters specifically for LongMemEval's temporal-reasoning category, which exercises date arithmetic.

Dropped facts. Across 10,273 distinctive-token facts, 18 are dropped (overall rate 0.18%). All 18 are from conv-43, which has the largest fact corpus (2,553 facts) and produced the *smallest* structured code (6,329 chars, vs. 12,222 for conv-30 with fewer facts). The pattern is consistent: when the structuring LLM is overloaded, it compresses by abstracting away atomic details rather than by losing structure or types. The 18 dropped facts are all from the same session (session_27) and concern minor personal-state details (e.g., "Tim finds learning German to be tough"). The notes bucket caught 423 facts that did not fit typed fields across the corpus—the safety net is doing its job.

Implications. Phase 2 has three failure axes: parse, type, content. The first two are clean on this benchmark (0/5 and 0/44 respectively). Content-loss is non-zero but small (0.18% overall on the audited five conversations) and concentrated on the longest input. This bounds the headline LOCOMO gap to Full Context: of the 1.0pp UaC-vs-Full Context gap on the full 10-conversation benchmark (Section 4), at most $\sim$0.2pp is explained by Phase 2 dropping facts the question needed; the remainder is in the retrieval layer (channel ablation in Section 4.6). At even larger scales the truncation behavior documented in Section E would dominate over the LLM-level compression we see here, which is why we flag hierarchical structuring as the right scaling strategy.

E Phase-2 Structuring Cost at Scale

Phase 2 is a single LLM call that regenerates structured Python from the entire fact corpus accumulated so far. To measure how its cost behaves outside the 19-session LOCOMO range, we run Phase 2 on synthetically lengthened corpora at $\{19, 50, 100, 200\}$ sessions, seeded from the actual conv-26 Phase 1 fact list (1,569 facts) and extended by appending shadowed-prefix duplicates of the same fact list to bring the total to $\sim$82 facts/session at each scale. The duplication is a worst-case stress test: a real long-running deployment would partially compress, so these numbers are an upper bound on what naive all-at-once Phase 2 needs.

Table 14. Phase-2 structuring on conv-26-seeded synthetic corpora at increasing session counts (Gemini 3 Flash, thinking budget 16,384). Input is hard-capped at 500K characters in our implementation; the cap binds at $n_{\text{sessions}} \geq 100$, so the entries at and above 100 are reading a truncated fact list. "Thoughts" is the Gemini thinking-token budget consumed; cost is \$0.30/M input +\$2.50/M output (treating thoughts as output).

n_{sess}	n_{facts}	In tok	Out tok	Thought tok	Wall (s)	USD
19	1,558	62,627	3,777	1,843	30.0	0.033
50	4,100	177,940	3,151	1,396	39.4	0.065
100	8,200	200,730	3,134	1,195	36.6	0.071
200	16,400	200,731	3,242	1,151	34.1	0.071

Three observations from Table 14:

(1) Output stays bounded. Output tokens hold at $\sim$3,100–3,800 across the full 10$\times$ range of input sizes, because the LLM increasingly groups and abstracts as input grows—a 200-session

corpus is structured into the same dataclass schema as a 50-session corpus, just with longer collections.

(2) Input is the only growing cost component, and our implementation caps it. At n=19 the input fits naturally (62K tokens, no truncation); by n=100 the 500K-character input cap engages and the input flattens at ∼200K tokens. Above that scale, our naive Phase 2 silently drops the tail of the fact list, which is exactly the limitation that motivates an incremental or hierarchical structuring strategy (Limitations, Section 5).

(3) Wall time is roughly constant at 30–40 seconds once the LLM's thinking step dominates. So in absolute terms, even on a 200-session deployment, Phase 2 is a sub-minute background operation, not a foreground latency item—but the truncation behavior means a production deployment shouldn't rely on naive regeneration past ∼100 sessions.

The clean linear regime (no truncation) is $n_{\text{sess}} \leq 50$: 0.78 m$ per session at n=19 and 1.30 m$ per session at n=50. Extrapolated naively, Phase 2 on a year of weekly sessions (∼52 sessions) would cost ∼$0.07 per regeneration. The cost is bounded by the LLM context, not by the number of facts.

F Real Cases: Conversation to Code to Constraint

This appendix walks through two end-to-end cases the main text only summarized: (i) a verbatim multi-session LOCOMO conversation distilled into a typed user state, and (ii) the conflicting wire-transfer scenario from our prototype, where two well-meaning family members give the agent contradictory destinations for the same transfer.

F.1 Case 1: LOCOMO `conv-30`, Jon and Gina

Listing 9 shows verbatim excerpts from three sessions of `conv-30` (Jon and Gina, January 2023). Listing 10 shows representative entries from the append-only fact list our Phase 1 extractor produces over those sessions (∼50 facts per session in the full run; we show the ones that anchor the most-cited LOCOMO QAs). Listing 11 shows the typed Python that Phase 2 generates from the complete fact corpus—absolute dates resolved, entities grouped, the `notes` field absorbing facts that do not fit typed fields. Listing 12 traces a real LOCOMO question through this representation.

```
SESSION 1 -- DATE: 2023-01-20 16:04
Jon:  Lost my job as a banker yesterday, so I'm gonna take a shot
      at starting my own business.
Gina: Sorry about your job Jon, but starting your own business
      sounds awesome! Unfortunately, I also lost my job at Door
      Dash this month.
Jon:  I'm starting a dance studio 'cause I'm passionate about
      dancing and it'd be great to share it with others.

SESSION 2 -- DATE: 2023-01-29 14:32
Gina: Just launched an ad campaign for my clothing store in hopes
      of growing the business.
Jon:  I'm on the hunt for the ideal spot for my dance studio...
      It's downtown which is awesome cuz it's easy to get to.
      Plus the natural light!
Jon:  I'm after Marley flooring, which is what dance studios
      usually use. Grippy but still lets you move.
```

```
SESSION 3 -- DATE: 2023-02-01 00:48
Gina: I emailed some wholesalers and one replied and said yes today!
      Now I can expand my clothing store.
Gina: Here's a peek at the space I designed. Cozy and inviting --
      perfect for customers to check out all the trendy pieces.
```

Listing 9. Verbatim excerpts from LOCOMO `conv-30` (Jon and Gina). Three sessions over twelve days.

```python
facts = [
    "[2023-01-19] Jon lost his job as a banker (one day before session 1)",
    "[2023-01-20] Jon plans to start his own business: a dance studio",
    "[2023-01-20] Jon's passion is dancing, contemporary is his favorite style",
    "[2023-01-20] Gina lost her job at Door Dash in January 2023",
    "[2023-01-29] Gina launched an ad campaign for her clothing store",
    "[2023-01-29] Jon is scouting downtown locations for the studio",
    "[2023-01-29] Jon wants Marley flooring (standard for dance studios)",
    "[2023-02-01] Gina secured a wholesaler relationship to expand the store",
    "[2023-02-01] Gina designed the store interior herself, choosing the decor",
    # ... ~140 more facts from later sessions ...
]
```

Listing 10. Phase 1 output – append-only fact list (representative entries from the full ∼150-fact corpus). Relative dates ("yesterday", "this month") are resolved against the session timestamp.

```python
@dataclass
class Person:
    name: str
    job_history: list["Job"] = field(default_factory=list)
    interests: list[str] = field(default_factory=list)
    notes: list[str] = field(default_factory=list)

@dataclass
class Job:
    title: str
    employer: str
    end_date: date | None = None
    reason_ended: str | None = None

@dataclass
class BusinessVenture:
    name: str
    type: str
    status: str  # "planning", "scouting", "operating"
    started: date
    requirements: list[str] = field(default_factory=list)
    notes: list[str] = field(default_factory=list)

```

```python
jon = Person(
    name="Jon",
    job_history=[Job(title="banker", employer="(unspecified)",
                     end_date=date(2023, 1, 19), reason_ended="laid off
                        ")],
    interests=["dancing", "contemporary dance"],
)

jon_studio = BusinessVenture(
    name="(unnamed dance studio)",
    type="dance studio", status="scouting",
    started=date(2023, 1, 20),
    requirements=["downtown location", "natural light",
                  "Marley flooring", "good bounce for dancers"],
    notes=["Jon's first business; motivated by passion for dance"],
)
```

Listing 11. Phase 2 output – typed Python state for Jon's side of `conv-30`, regenerated from the complete fact corpus. Gina's parallel module is structurally identical and is omitted for space.

```python
# Question (LOCOMO category 2, temporal):
#   "When did Jon lose his job as a banker?"
#
# Retrieval (Section 3.5):
#   [STATE]    jon.job_history[0].end_date == date(2023, 1, 19)
#   [FACTS]    "[2023-01-19] Jon lost his job as a banker..."
#   [ARCHIVE]  Session 1, Jon turn 1: "Lost my job as a banker
   yesterday"
#
# Answer: "January 19, 2023"  -- judged CORRECT.
# Note: Mem0 on the same question returns "yesterday" because its
#       extracted memory preserves the relative-date phrasing without
#       resolving it against the session timestamp.
```

Listing 12. A real LOCOMO QA against this state. The judge accepts both "19 January, 2023" and the typed form below as correct.

The key observation from Listing 12 is the failure mode that motivates Phase 1's date-resolution step: "yesterday" is a perfectly recallable string, but it is the *wrong* answer. Resolving relative dates at extraction time, against the session timestamp, is one of the simplest interventions in the pipeline and accounts for a large share of the gap between UaC and Mem0 on LOCOMO's temporal-reasoning subset.

F.2 Case 2: Conflicting Wire Transfers

Listing 13 shows the typed state our prototype maintains for Jessica Thompson's finance domain after a single ambiguous week: her mother and her husband each told the agent a different destination institution for the same $15,000 wire. Without an executable cross-check the agent would dutifully act on whichever instruction it heard most recently—and either send the money to the wrong bank or escalate the conflict only after the recipient complains. Listing 14 shows the constraint that fires; Listing 15 shows the resulting alert.

```
pending_transfers = [
    WireTransfer(
        amount=15000.00, currency="USD",
        recipient_name="Patricia Williams",
        destination_institution="Bank of America",
        destination_account_last_four="3310",
        purpose="Gift to mother",
        requested_by="Patricia Williams (mother)",
        requested_date=date(2025, 1, 8),
        notes="Mom called and asked Jessica to send to her BofA account
            ",
    ),
    WireTransfer(
        amount=15000.00, currency="USD",
        recipient_name="Patricia Williams",
        destination_institution="Wells Fargo",
        destination_account_last_four="6654",
        purpose="Gift to mother",
        requested_by="James Thompson (husband)",
        requested_date=date(2025, 1, 9),
        notes="James said Patricia's account is at Wells Fargo, not
            BofA",
    ),
]
```

Listing 13. Excerpt from `domains/finance/state.py`. Two pending transfers, same amount, same recipient, different destinations, different requesters.

```
def check_financial_authorization(transfers: list[WireTransfer]) ->
    list[Alert]:
    groups: dict[tuple, list[WireTransfer]] = {}
    for t in transfers:
        if t.status != "pending":
            continue
        groups.setdefault((t.recipient_name, t.amount, t.purpose), []).
            append(t)

    alerts = []
    for (recipient, amount, _), group in groups.items():
        destinations = {(t.destination_institution,
                         t.destination_account_last_four) for t in
                            group}
        if len(destinations) > 1:
            descs = [f"{t.destination_institution} (****"
                     f"{t.destination_account_last_four}) per {t.
                        requested_by}"
                     for t in group]
            alerts.append(Alert(
                severity="critical", domain="finance",
                message=(f"CONFLICTING wire transfer instructions for "
                         f"${amount:,.2f} to {recipient}: " + " vs. ".
                            join(descs) +
```

```
                            ". Verify with the account holder before
                               sending.")))
    return alerts
```

Listing 14. Constraint from `constraints/financial_authorization.py`: groups pending transfers by (recipient, amount, purpose) and flags any group with >1 distinct destination.

```
[CRITICAL/finance]  CONFLICTING wire transfer instructions for
$15,000.00 to Patricia Williams: Bank of America (****3310) per
Patricia Williams (mother) vs. Wells Fargo (****6654) per James
Thompson (husband). Verify with the account holder before sending.
```

Listing 15. Generated alert. The agent now blocks the transfer and asks Jessica to verify, instead of acting on whichever instruction it heard last.

F.3 Case 3: Analytical Query over the User's History

The analytical-inference benchmark (Section 4.3) measures the gap between code-executable and retrieval-only representations on aggregate questions. This subsection traces one such question end-to-end against UaC and a retrieval baseline so the structural reason for the gap is concrete.

Listing 16 shows the typed state Phase 2 produces for a synthetic user with 200 dining records over two years. Listing 17 shows how UaC answers the question *"what was my average dinner spend in 2024 by cuisine, top 3?"* —the LLM emits one pandas-style aggregation against the typed records, the Python REPL executes it deterministically, and the answer is correct by construction. Listing 18 shows how a retrieval-based system (MemMachine-style sentence-level retrieval with ±3 contextual expansion) fails on the same question: top-k retrieval surfaces a sample of the relevant records, the LLM averages over the sample and reports the wrong number with the right shape.

```
@dataclass
class Meal:
    date: date
    meal_type: str          # "breakfast", "lunch", "dinner"
    cuisine: str            # "Italian", "Japanese", "Mexican", ...
    venue: str
    party_size: int
    total_usd: float

meals = [
    Meal(date(2024, 1, 4),  "dinner", "Japanese", "Nobu",        2,
        184.50),
    Meal(date(2024, 1, 6),  "lunch",  "Mexican",  "El Farolito",1,
        14.25),
    Meal(date(2024, 1, 12), "dinner", "Italian",  "Frances",     4,
        312.00),
    Meal(date(2024, 1, 18), "dinner", "Japanese", "Kabuto",      2,
        96.40),
    # ... 196 more records spanning 2024-01-01 to 2025-12-31 ...
]
```

Listing 16. Excerpt from the synthetic dining state (N=200 records, abbreviated to 4 for space). The full state is a list of `Meal` dataclass instances generated by Phase 2 from atomic fact extractions.

```
# Question: "What was my average dinner spend in 2024 by cuisine, top 3?"
# UaC -- LLM emits this one expression against the typed state:
>>> from statistics import mean
>>> from collections import defaultdict
>>> by_cuisine = defaultdict(list)
>>> for m in meals:
...     if m.meal_type == "dinner" and m.date.year == 2024:
...         by_cuisine[m.cuisine].append(m.total_usd)
>>> sorted(((c, round(mean(v), 2), len(v)) for c, v in by_cuisine.items()),
...        key=lambda x: -x[1])[:3]
[('Italian',  187.45, 31),
 ('Japanese', 152.30, 28),
 ('French',   141.80, 14)]

# Answer surfaced to user: "Italian $187.45 (31 meals), Japanese $152.30
# (28 meals), French $141.80 (14 meals)."  -- exact-match CORRECT.
```

Listing 17. UaC answer trace. The LLM writes one line of Python against the typed state; the Python REPL executes it. Total LLM tokens: ∼200 in, 60 out. Result is deterministic given the state.

```
# Same question, MemMachine pipeline:
#   nucleus retrieval (k=30) on "average dinner spend 2024 by cuisine"
#   returns 30 sentences out of ~120 relevant records (~25% coverage).
#   LLM reasons over those 30 sentences:
# Answer: "Italian ~$210, Japanese ~$145, Mexican ~$48."  -- WRONG.
#   * Italian over-estimated: the 30 retrieved records over-sampled high-end
#     Italian (Nobu, Frances) and missed cheaper trattorias.
#   * Mexican is in the top-3 only because lunches got included; dinner-only
#     filter cannot be enforced on a free-text retrieval result.
#   * Counts not reported because top-30 does not equal the true population.
#
# Top-k retrieval is structurally lossy for aggregation: the answer
# requires *all* relevant records, not the *most relevant* ones.
```

Listing 18. MemMachine answer trace on the same question. Top-30 dense retrieval surfaces only a sample of 2024 dinner records; the LLM averages over what it sees and reports a confident but wrong number.

The key generalization: *retrieval ranks; aggregation enumerates*. Any question whose answer depends on every relevant record (counts, sums, group-bys, time-window filters, top-k within a population) is structurally outside what top-k retrieval can solve. UaC's typed state lifts the question from natural language to one line of Python, where the interpreter handles enumeration for free. The 99% vs. 43% gap on the analytical benchmark (Table 3) is this same trace, repeated across 100 questions.

F.4 Why these cases matter beyond the benchmark numbers

The drug-allergy interaction in Listings 2–5 and the wire-transfer conflict above are both real-deployment scenarios where the user is in genuine danger from agent error. They share a structure that no LOCOMO-style QA captures: *the user never asks the question*. They depend on the agent noticing a constraint violation across facts surfaced in different sessions, by different people, possibly months apart. This is exactly the regime where a representation that supports executable boolean checks beats a representation that supports only similarity-ranked retrieval, and is why we built the Active Service benchmark (Section 4.4) as a distinct evaluation axis. The headline numbers—100% on standard scenarios, 85% on hard—are read most fairly with these scenarios in mind: each missed alert is a real harm avoided, not a ranking-list position.

References

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. ECAI 2025.

Pengfei Du. Memory for autonomous llm agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026.

Jizhan Fang, Xinle Deng, Haoming Xu, Ziyan Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. Lightmem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*, 2025. ICLR 2026.

Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026.

Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. Evermemos: A self-organizing memory operating system for structured long-horizon reasoning. *arXiv preprint arXiv:2601.02163*, 2026.

Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*, 2025a. ICLR 2026; introduces MemoryAgentBench.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*, 2025b.

Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. Magma: A multi-graph based agentic memory architecture for ai agents. *arXiv preprint arXiv:2601.03236*, 2026. ACL 2026 Main Conference.

LangChain Team. Langmem sdk for agent long-term memory. https://github.com/langchain-ai/langmem, 2025. Open-source software.

Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025.

Yifei Li, Weidong Guo, Lingling Zhang, Rongman Xu, Muye Huang, Hui Liu, Lijiao Xu, Yu Xu, and Jun Liu. Locomo-plus: Beyond-factual cognitive memory evaluation framework for llm agents. *arXiv preprint arXiv:2602.10715*, 2026.

Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhiqiang Yin, Qingchen Yu, Bo Tang, Hongkang Yang, Zhi-Qin John Xu, and Feiyu Xiong. Memos: An operating system for memory-augmented generation (mag) in large language models. *arXiv preprint arXiv:2505.22101*, 2025.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024. ACL 2024.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Daivik Patel and Shrenik Patel. Engram: Effective, lightweight memory orchestration for conversational agents. *arXiv preprint arXiv:2511.12960*, 2025.

Sidharth Pulipaka, Oliver Chen, Manas Sharma, Taaha S Bajwa, Vyas Raina, and Ivaxi Sheth. Persistbench: When should long-term memories be forgotten by llms? *arXiv preprint arXiv:2602.01146*, 2026.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

Richard Sutton. The bitter lesson. http://www.incompleteideas.net/IncIdeas/BitterLesson.html, 2019. Blog post, March 13, 2019.

Yuanbo Tang, Huaze Tang, Tingyu Cao, Lam Nguyen, Anping Zhang, Xinwen Cao, Chunkang Liu, Wenbo Ding, and Yang Li. Proagentbench: Evaluating llm agents for proactive assistance with real-world data. *arXiv preprint arXiv:2602.04482*, 2026.

Shu Wang, Edwin Yu, Oscar Love, Tom Zhang, Tom Wong, Steve Scargall, and Charles Fan. Memmachine: A ground-truth-preserving memory system for personalized ai agents. *arXiv preprint arXiv:2604.04853*, 2026.

Yu Wang, Ryuichi Takanobu, Zhiqi Liang, Yuzhen Mao, Yuanzhe Hu, Julian McAuley, and Xiaojian Wu. Mem-α: Learning memory construction via reinforcement learning. *arXiv preprint arXiv:2509.25911*, 2025.

Lei Wei, Xiao Peng, Xu Dong, Niantao Xie, and Bin Wang. Fademem: Biologically-inspired forgetting for efficient agent memory. *arXiv preprint arXiv:2601.18642*, 2026.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2025. ICLR 2025.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025. NeurIPS 2025.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.

Chang Yang, Chuang Zhou, Yilin Xiao, Su Dong, Luyao Zhuang, Yujing Zhang, Zhu Wang, Zijin Hong, Zheng Yuan, Zhishang Xiang, Shengyuan Chen, Huachi Zhou, Qinggang Zhang, Ninghao Liu, Jinsong Su, Xinrun Wang, Yi Chang, and Xiao Huang. Graph-based agent memory: Taxonomy, techniques, and applications. *arXiv preprint arXiv:2602.05665*, 2026.

Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026.